AI 기초프로젝트 레벨 2 결과보고서

학년/학번	2/20231286	이름	황재욱
레벨	2	문제번호	5
문제명	POS(Point-Of-Sale) 시스템		

※ 구현 결과 요약

필수 요구사항 25 개와 직접 정의한 확장 기능 19 개를 모두 구현했으며, 확장 기능의 핵심은 **실제 식당 영업 하루를 처음부터 끝까지 처리하는 흐름**(시재 개시 → 주문·옵션 → 주방 주문서 → 결제·부분 환불 → 시재 정산 → 일일 마감)이다. 업무 규칙은 단위 테스트 59 개로 검증했고, Windows 실행 파일(exe)로 배포한다.

1) 기본 기능

필요한 기능	완성 여부	설명
가게명·주제 자유 설정	o	분식집 "금오분식". 상호·주소·계좌는 `config.py` 한 곳에서 변경
테이블별 주문, 메뉴별 단가·수량 기록	o	주문 항목에 메뉴명·단가·원가·할인율·수량을 주문 시점 값으로 저장(이후 가격이 바뀌어도 과거 매출 불변)
주문 추가·변경·삭제	o	메뉴 버튼 추가, +1/-1/수량 입력/항목 삭제, 주문 전체 취소
전체 메뉴 / 총액만 영수증 선택	o	결제 완료 후 상세·간이·출력 안 함 선택. 파일 저장과 기본 프린터 인쇄
지불 방법 선택, 현금 거스름돈	o	현금·카드·간편결제·계좌이체. 받은 금액 입력 시 거스름돈 실시간 계산
전체·메뉴별 매출, 메뉴별 주문 수 통계	o	매출 요약 카드, 메뉴별 판매수량·주문 수·매출·원가·이익
사용자 모드 / 관리자 모드	o	관리자는 비밀번호(F12). 사용자 탭 5 개, 관리자 탭 9 개 추가
주문 연동 재고 차감, 부족 경고	o	메뉴별 레시피(재료 소모량)로 차감, 취소 시 복구, 기준 이하 경고
일·주·월·연·기간 통계	o	오늘/이번 주/이번 달/올해/최근 30 일/직접 입력, 추이 그래프 4 단위
직원 근무 기록과 급여 계산	o	출퇴근 버튼, 월 급여(기본급·연장수당·주휴수당)
GUI 환경	o	Python Tkinter. 막대 그래프 직접 구현

필요한 기능	완성 여부	설명
회원 등록, 포인트 적립·사용	o	휴대폰 번호 회원, 등급별 1~3% 적립, 1,000P 이상 사용
메뉴별·시간대별·쿠폰 할인, 할인 전/후 금액	o	메뉴 → 시간대 → 쿠폰 → 포인트 순서 적용, 결제창·영수증에 단계별 표시
분할 결제(메뉴별/금액별)	o	금액 분할(1/N 계산), 메뉴·수량 선택 분할, 결제수단 혼합
테이블 병합·분할	o	빈 테이블 이동, 두 테이블 합치기, 메뉴·수량 골라 나누기
포장·매장 구분, 유형별 통계	o	포장은 별도 목록·주문번호·픽업 완료. 유형별 건수·매출·비율
접수·완료 시각, 평균 처리·조리 시간	o	주방 화면에서 조리 완료 → 주문 처리 시간과 메뉴별 평균 조리 시간
판매 완료 주문 취소·환불, 통계 반영	o	전체 환불과 메뉴 단위 부분 환불. 환불일 매출에서 차감
메뉴 인기 순위(판매량·매출)	o	판매수량/매출/이익 정렬, Top 10 그래프
시간대별 주문 수·매출, 피크 시간	o	24 시간 막대그래프 2 종, 피크 시간대 강조
품절 자동 처리, 재입고 시 판매 재개	o	만들 수 있는 수량이 0 이면 버튼 비활성화, 입고 시 자동 해제
메뉴별 원가·이익, 전체 순이익	o	메뉴 이익·이익률, 순이익 = 실결제액 – 원가
일일 마감(건수·매출·결제수단·취소/환불)	o	마감 보고서 미리보기·저장·인쇄, 시재 정산 내역 포함
카드·현금·간편결제 비율·매출	o	결제수단별 건수·순금액(결제–환불)·비율
예약 주문, 예약 시각 확인	o	예약 10 분 전 팝업 알림, 한 번에 실제 주문으로 전환

2) 확장 기능

필요한 기능	완성 여부	설명
시재(금전함 현금) 관리	o	개시·입출금·권종별 중간 점검·정산. 과부족 사유 필수, 1 만원 이상은 관리자만 정산
메뉴 옵션	o	맵기(하나 선택), 곱빼기·토핑(복수 선택) 추가금. 옵션 재료도 재고 차감
요청사항 메모	o	"덜 맵게" 등 메뉴별 메모. 주방 화면·주문서에 표시
주방 화면(KDS)과 주방 주문서	o	조리 대기열, 15 분 지연 강조. 주문서는 마지막 출력 이후 추가·취소분만 인쇄

필요한 기능	완성 여부	설명
계좌이체 결제	o	입금 계좌 안내, 영수증·통계·마감 반영
부분 환불	o	메뉴·수량 단위. 할인 비율 배분, 결제수단별 환불 배분, 포인트 비율 회수
기간 비교 통계	o	전월·전년·지난달 같은 날짜·직전 기간과 지표·메뉴별 증감
인쇄	o	OS 기본 프린터로 영수증·주문서·마감·정산서 인쇄. 실패 시 파일 저장
회원 등급제	o	누적 결제액으로 일반/실버/골드 자동 등급, 적립률 차등
N 명 균등 분할 계산	o	남은 금액 ÷ 남은 인원, 10 원 단위 올림
데이터 영속 저장(SQLite)	o	25 개 테이블. 껐다 켜도 진행 중 주문·통계 유지
DB 백업·복원	o	시각별 백업, 복원 전 자동 백업, 파일 무결성 검사
DB 자동 업그레이드	o	이전 버전 `pos.db`에 새 컬럼·테이블 자동 추가
작업(감사) 로그	o	결제·환불·재고·로그인·마감 등 모드와 함께 기록
CSV 내보내기	o	통계·급여 명세를 엑셀용 CSV(UTF-8 BOM)로 저장
샘플 데이터 생성기	o	과거 판매·근태·시재 데이터 생성(통계 시연용)
Windows 실행 파일(exe)	o	PyInstaller 단일 exe, 데이터는 exe 옆 폴더, 오류는 `data\error.log`
예약 노쇼·취소 처리	o	대기 → 주문 전환/취소/노쇼
단위 테스트	o	업무 규칙 59 개 테스트, 고정 시계로 시간 의존 로직 검증

3) 직접 정의한 제한사항(업무 규칙)

문제에서 "제한사항은 개별적으로 정한다"고 했으므로 실제 매장 운영을 기준으로 아래 규칙을 정했다. 모든 수치는 `config.py` 상수로 관리하며, 어기는 입력은 안내 메시지와 함께 거부한다.

분류	주요 규칙
주문	메뉴 1~99 개, 주문당 50 줄, 테이블 최대 30 개, 한 테이블에 진행 주문 1 개, 재고 부족 메뉴 주문 불가, 결제가 시작된 주문은 변경 불가
옵션·요청사항	같은 그룹 옵션은 하나만, 추가금 0~10,000 원(100 원 단위), 요청사항 40 자·조리 전만 수정
할인	메뉴 → 시간대(가장 큰 1 개) → 쿠폰(1 장) → 포인트 순서, 10 원 미만 절사, 메뉴 할인 ≤ 50%, 시간대 ≤ 30%

분류	주요 규칙
결제	분할 최대 10 회, 첫 결제 시 금액 확정(쿠폰·포인트 차감), 현금은 받은 금액 ≥ 결제 금액
회원·포인트	번호 중복 불가, 일반 1%/실버 2%/골드 3%, 1,000P 이상 100P 단위 사용
취소·환불	결제 전 취소는 사용자, 결제 후 환불은 관리자만(사유 필수). 환불액은 환불일 매출에서 차감
시재	시재 개시 중에만 현금 결제·환불, 금전함보다 많이 출금·환불 불가, 과부족 사유 필수, 1 만원 이상 과부족은 관리자만 정산
직원·급여	2026 년 최저시급 10,320 원 이상, 1 회 근무 최대 16 시간, 근무 시간 중복 불가
예약	30 분 이후만 예약, 같은 테이블 앞뒤 1 시간 중복 불가, 10 분 전 알림
마감·보안	미결제 주문 0 건·시재 정산 완료 시에만 마감, 비밀번호 숫자 4~8 자리·해시 저장·3 회 실패 시 30 초 잠금

1. 서론

이 프로젝트는 식당에서 쓰는 POS 프로그램을 Python 으로 만들고, 문제에 주어진 요구사항을 넘어 **실제 하루 영업을 끝까지 처리할 수 있는 수준**까지 다듬는 것을 목표로 한다.

1.1 개발 목표

1. 문제에서 요구한 기능 25 개를 잘못된 입력에서도 프로그램이 멈추지 않도록 구현한다.
2. 제한사항(업무 규칙)을 직접 정하고, 규칙마다 자동 테스트로 지켜지는지 확인한다.
3. 현장에서 필요한 확장 기능(시재, 옵션, 주방 주문서, 부분 환불 등)을 추가하고 exe 로 배포한다.

1.2 필요성 및 개발 배경

식당·카페·주점은 주문, 결제, 재고, 직원, 매출을 동시에 관리해야 한다. 종이 장부나 엑셀로는 테이블별 주문 변경, 분할 결제, 환불 후 매출 정정, 마감 정산을 실수 없이 처리하기 어렵다. 이 프로젝트는 POS 의 핵심 업무 규칙을 직접 설계하면서 "돈과 재고 숫자가 언제나 맞아야 한다"는 요구를 프로그램으로 보장하는 방법을 익히는 데 초점을 맞췄다.

개발 중에는 용도도 함께 검토했다. 편의점 모드를 추가하는 방안을 비교했지만, 바코드·유통기한 중심의 편의점 기능은 과제의 필수 요구사항(테이블, 조리 시간, 매장/포장)과 맞지 않아 **식당 전용으로 범위를 확정**했다.

1.3 관련 기술 동향 및 선행 기술

구분	기술	이 프로젝트와의 관계
상용 POS	클라우드 POS(포스·키오스크·태블릿 연동)	여러 기기가 같은 데이터를 공유. 이 프로젝트는 단일 PC 대상

구분	기술	이 프로젝트와의 관계
상용 POS	테이블 오더·키오스크	손님이 직접 주문. 이 프로젝트는 직원 입력 방식
주방 연동	KDS(주방 디스플레이)	주방 화면 탭과 주방 주문서로 구현
결제	VAN 사 카드 승인 연동	인증이 필요해 범위 밖. 결제수단만 기록
인쇄	ESC/POS 영수증 프린터 명령	프린터 모델에 종속. 대신 OS 기본 프린터 사용
저장소	SQLite(임베디드 DB)	설치 없는 파일 DB. 트랜잭션·집계 쿼리 사용
노무	근로기준법 주휴·연장수당	급여 계산 규칙에 반영(2026 최저시급)
배포	PyInstaller	Python 없이 실행되는 exe 생성

1.4 문제 분석

요구사항을 정리하면서 문제 본문만으로는 정해지지 않는 세 가지를 찾았고, 대안을 비교해 결정했다.

① 환불이 "통계에 자동 반영"되는 시점이 정해져 있지 않다. 어제 판매한 주문을 오늘 환불하면 어제 매출을 고칠지, 오늘 매출에서 뺄지에 따라 이미 출력한 마감 보고서와 숫자가 어긋날 수 있다.

대안	장점	단점	선택
A. 환불한 날 매출에서 차감	과거 마감 보고서와 항상 일치. 실제 현금 흐름과 같음	결제일과 환불일 통계를 함께 봐야 함	채택
B. 결제한 날 매출을 수정	주문 단위로 보면 단순	이미 마감한 날의 숫자가 바뀜	기각
C. 환불 주문을 통계에서 제외만	구현이 가장 쉬움	환불액이 언제 빠졌는지 알 수 없음	기각

② 분할 결제와 주문 할인을 함께 쓰면 금액을 어떻게 나눌지 정해져 있지 않다. 메뉴별로 나눠 낼 때 시간대·쿠폰·포인트 할인은 특정 메뉴가 아니라 주문 전체에 걸린다.

대안	장점	단점	선택
A. 금액 비율로 배분, 10 원 단위 반올림, 마지막 결제가 끝전 정산	누구도 손해 보지 않고 합계가 정확	계산이 조금 복잡	채택
B. 할인을 첫 결제자에게 몰기	단순	먼저 낸 사람만 이득	기각
C. 분할 결제 시 할인 금지	가장 쉬움	요구사항(할인+분할)을 함께 못 씀	기각

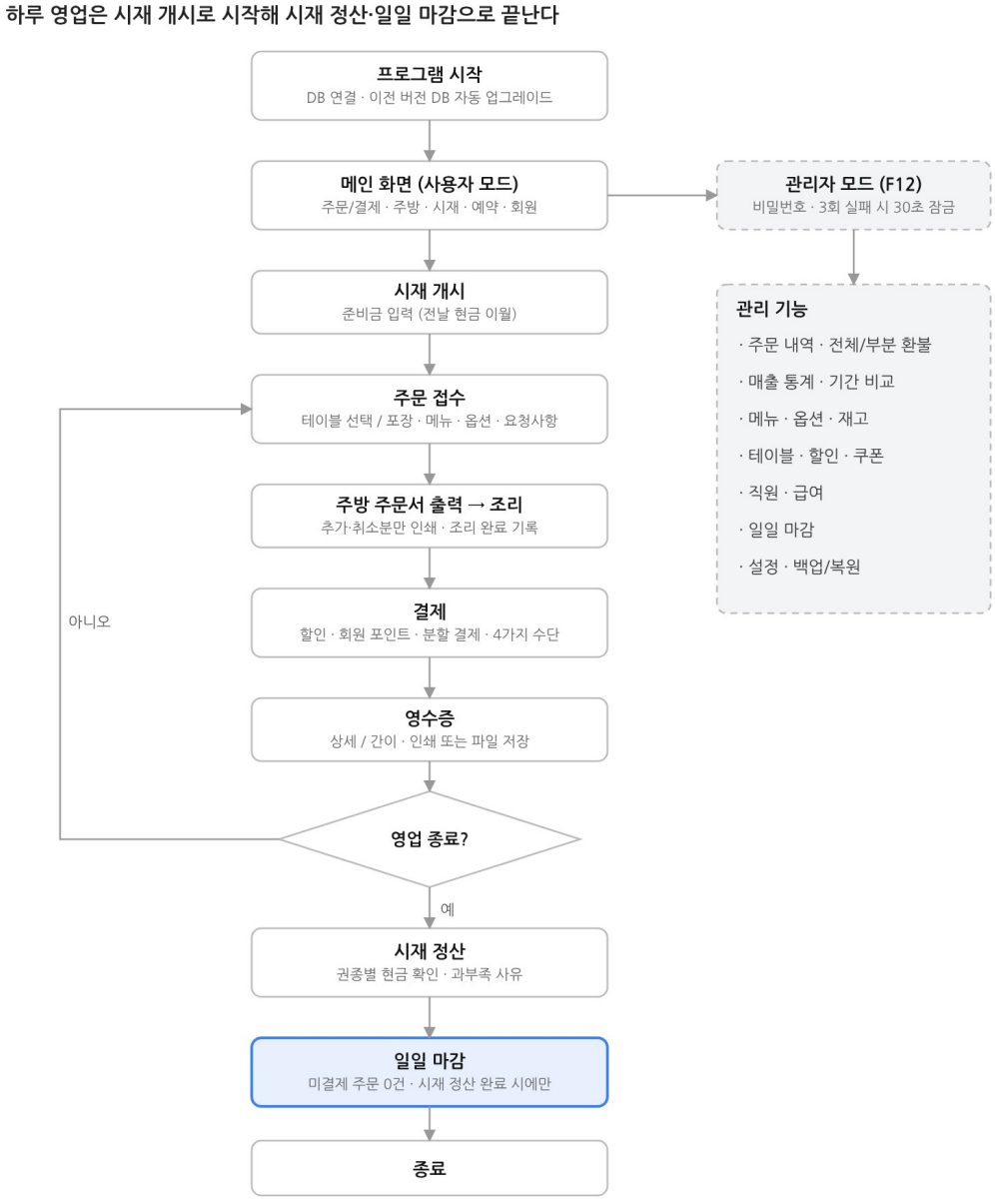
③ "메뉴 주문에 따라 식재료 재고를 변경"하려면 메뉴와 재료의 관계가 필요하다. 메뉴 개수를 재고로 두면 떡볶이와 라볶이처럼 같은 재료(떡)를 쓰는 메뉴를 표현할 수 없다. 그래서 메뉴마다 **레시피(재료 소모량)** 를 두고, 주문 가능 수량을 재료별 `재고 ÷ 사용량`의 최솟값으로 계산했다. 이 구조 덕분에 옵션(치즈 추가 → 치즈 1 장)도 같은 방식으로 재고를 차감한다.

> ***AI 활용*** *— 프롬프트 "이번에 학교 과제로 pos 기기 프로그램 만들껀데 14 페이지 부터 15 페이지의 내용으로 만들꺼야 헤당 이미지의 내용을 따를 필요는 없고 제한 사항이랑 확장 내용은 알아서 추가하는걸로 해보자"에 대해 AI(Claude)가 요구사항 25 개를 정리하고 제한사항·확장 기능을 정의했다. 용도 검토는 "편의점에서 사용할려면 어떤 기능들을 추가해야할까", "음식점 용도와 편의점 모드를 같이 가져갈때 이점과 편의점 모드만 했을때의 이점" 질문으로 진행했다. 전체 프롬프트 목록은 12 장에 있다.*

2. 실행 흐름도

프로그램은 탭으로 나뉜 하나의 창에서 동작하며, 하루 영업은 시재 개시로 시작해 시재 정산과 일일 마감으로 끝난다.

2.1 전체 흐름

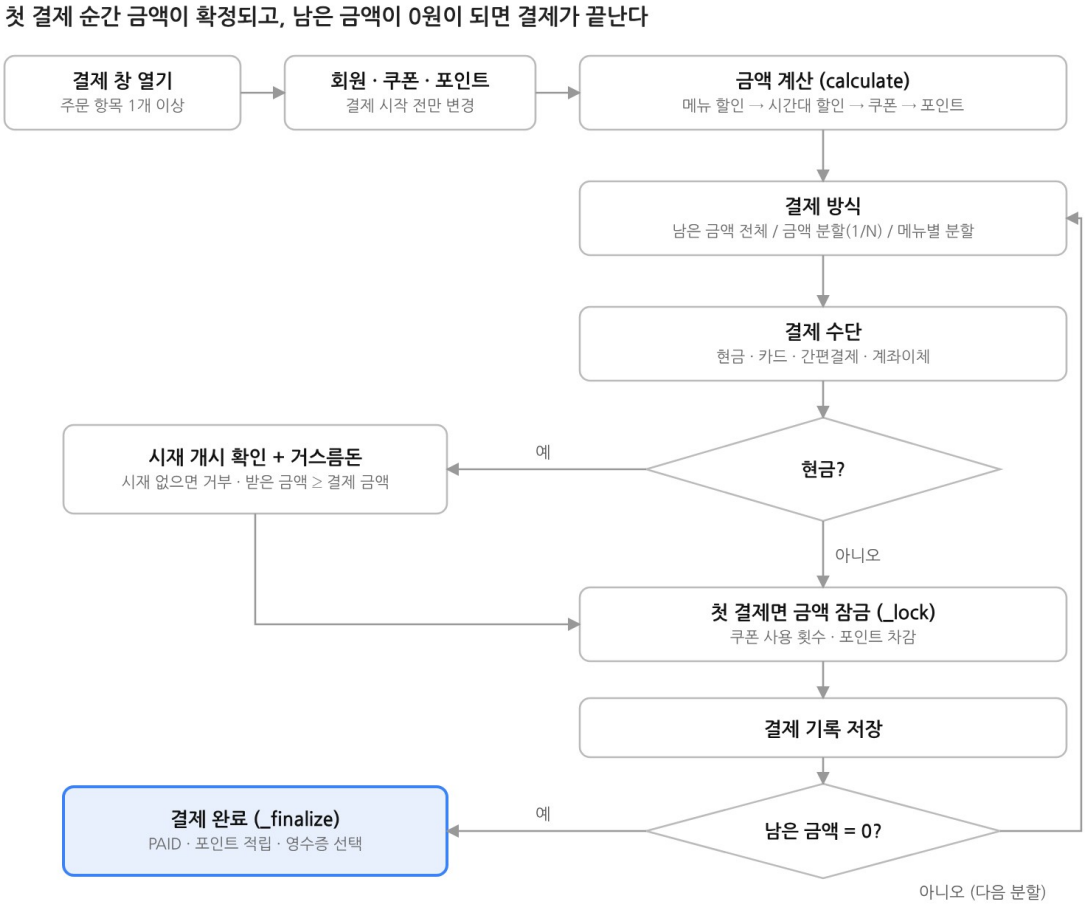


전체 실행 흐름 · 하루 영업 순서

관리자 기능은 F12 와 비밀번호로 언제든 열 수 있다. 업무 규칙을 어기는 조작은 화면이 멈추지 않고 안내 창만 띄운 뒤 원래 화면으로 돌아온다.

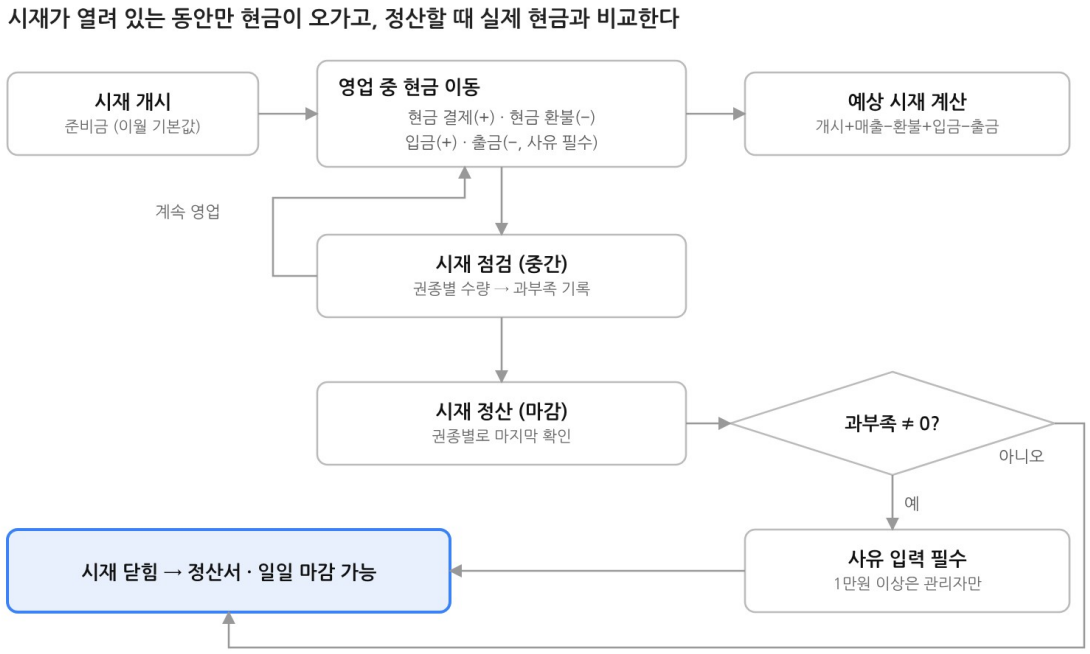
2.2 입력 → 처리 → 출력: 주문과 결제

1. **입력:** 테이블(또는 포장)을 고르고 메뉴를 누른다. 옵션이 있는 메뉴는 옵션 창에서 맵기·추가 선택·수량·요청사항을 입력한다.
2. **처리:** 재고를 확인하고 재료(옵션 재료 포함)를 차감한다. 같은 메뉴·옵션·요청사항이면 수량을 합친다.
3. **주방 전달:** "주방 주문서 출력"을 누르면 마지막 출력 이후 추가·취소된 수량만 인쇄한다.
4. **결제:** 할인을 순서대로 계산하고, 첫 결제 순간 금액을 확정한다. 남은 금액이 0 원이 될 때까지 분할 결제한다.
5. **출력:** 결제가 끝나면 포인트를 적립하고 영수증(상세/간이)을 화면·파일·프린터로 출력한다.



결제 처리 흐름

2.3 입력 → 처리 → 출력: 시재 정산



시재(금전함 현금) 처리 흐름

현금 결제와 현금 환불은 그때 열려 있는 시재에 기록되고, 정산할 때 권종별로 센 실제 현금과 예상 시재를 비교한다. 시재가 정산되어야 일일 마감을 할 수 있다.

> ***AI 활용*** *— 흐름도는 프롬프트 "...지금 어떤게 되어 있는지랑 필수 과제 + 확장 과제 어떻게 되어 있는지 정리해주고 보고서 양식은 올린 파일과 동일하게 진행"에 대해 AI 가 실제 코드의 분기 구조를 바탕으로 SVG 도식으로 작성했다.*

3. 시스템 구성

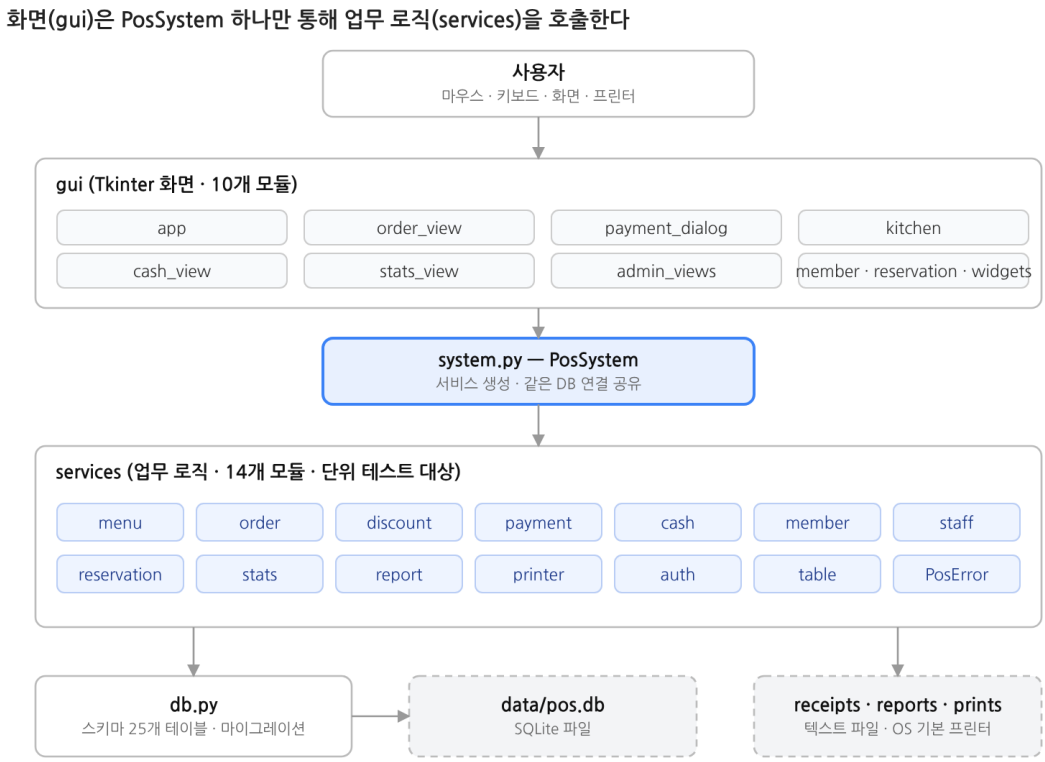
일반 Windows PC 한 대에서 동작하는 단독 프로그램이며 네트워크는 사용하지 않는다. Python 표준 라이브러리만 사용하므로 별도 설치가 필요 없고, 배포는 exe 하나로 한다.

3.1 하드웨어 및 소프트웨어 환경

구분	내용
하드웨어	일반 PC(키보드·마우스, 모니터). 영수증 프린터는 선택(OS 기본 프린터로 지정)
언어	Python 3.9 이상(개발·테스트 3.12)
GUI	Tkinter / ttk (표준 라이브러리)
데이터베이스	SQLite 3 (표준 라이브러리 sqlite3), 파일 `data/pos.db`
외부 라이브러리	실행에는 없음. exe 빌드에만 PyInstaller
실행 환경	Windows 10/11(exe), Python 이 있으면 macOS·Linux 도 가능

구분	내용
데이터 파일	`data/`(DB·오류 로그), `receipts/`, `reports/`, `prints/`, `exports/`, `backups/`
네트워크	사용하지 않음

3.2 모듈 구성



모듈 구성도 · 화면(gui) – 파사드(PosSystem) – 업무 로직(services) – DB

화면 모듈은 `PosSystem` 하나만 보고 업무 로직을 호출한다. 돈·재고 계산은 모두 `services`에 있으므로 화면 없이도 테스트할 수 있고, 화면을 바꿔도 로직은 그대로 쓴다.

4. 설계구성요소 및 설계제한요소

설계는 "어떤 조작에도 프로그램이 멈추지 않는다"와 "돈과 재고 숫자는 어떤 순서로 조작해도 맞는다" 두 가지를 기준으로 진행했다.

목표 설정	합성	분석	구현/제작	시험/평가	결과 도출	성능	규격/표준	경제성	미학	신뢰성	안정성/내구성	환경
•	•	•	•	•	•	•	•	•	•	•	•	

1) 설계구성요소

① **목표 설정** — 필수 요구사항 25 개를 기능 목록으로 쪼개고, 확장 목표를 "식당 하루 영업 전체 처리"로 정했다. 편의점 모드, 발주 추천도 검토했으나 과제 요구사항과 맞지 않거나(편의점) 추정 결과가 불안정해(발주 추천) 제외했다.

② **합성** — 주문·결제·재고·회원·직원·통계·시재를 하나의 DB 와 `PosSystem` 으로 묶어, 한 번의 결제가 재고·포인트·쿠폰·시재·통계에 동시에 반영되게 했다.

③ **분석** — 1.4 절의 세 가지 모호한 점(환불 반영 시점, 분할+할인, 재고 구조)을 찾아 대안을 비교했다.

④ **구현/제작** — 주요 결정과 비교한 대안은 다음과 같다.

결정 사항	채택	비교한 대안과 기각 이유
언어·GUI	Python + Tkinter	C/Java: GUI·DB 에 외부 라이브러리가 필요하고 개발량이 큼
저장소	SQLite	JSON/CSV 파일: 트랜잭션이 없어 결제 중 오류 시 일부만 저장될 위험, 집계가 느림
코드 구조	업무 로직(services)과 화면(gui) 분리	화면에 로직 작성: 자동 테스트 불가
금액 자료형	정수(원 단위)	실수(float): 0.1 원 단위 오차 누적
주문 항목 가격	주문 시점 값 저장(스냅샷)	메뉴 테이블 참조: 가격을 바꾸면 과거 매출이 변함
삭제 방식	비활성화(숨김)	실제 삭제: 과거 주문·급여 기록이 깨짐
결제 중 수정	첫 결제 시 금액 잠금	계속 수정 허용: 이미 받은 돈과 주문 금액이 어긋남
환불 기록	환불 테이블(전체·부분 통합)	결제 행에 환불 표시만: 부분 환불·수단별 환불 표현 불가
시재 연결	결제·환불 행에 시재 id 저장	시간 구간으로 계산: 개시·정산 경계 시각에 오차
인쇄	OS 기본 프린터 + 실패 시 파일	ESC/POS 직접 제어: 프린터 모델 종속, 외부 라이브러리 필요
배포	PyInstaller 단일 exe	폴더 배포: 파일 누락 위험, 사용자가 Python 설치 필요
DB 변경	실행 시 자동 업그레이드	새 DB 강제: 기존 영업 데이터 손실

⑤ **시험/평가** — 시간을 고정한 가짜 시계(Clock)로 시간대 할인·예약·근무시간처럼 시간에 따라 달라지는 로직을 테스트했다. 단위 테스트 59 개, GUI 자동 조작 시나리오 4 종, exe 실행 테스트를 단계마다 반복했다(9 장).

⑥ **결과 도출** — 필수 기능 25 개와 확장 기능 19 개를 구현했고, 90 일치 샘플 데이터(주문 2,856 건)에서 통계 7 종을 0.18 초에 계산했다.

2) 설계제한요소

① **성능** — 주문·결제 처리는 주문 항목 수 n 에 대해 O(n)이다. 통계는 SQLite 의 인덱스(결제 시각·상태)와 GROUP BY 로 계산해 90 일치 6,832 개 주문 항목에서도 0.2 초 안에 끝난다.

② **규격/표준** — Python 표준 라이브러리만 사용해 Python 3.9 이상이 설치된 Windows·macOS·Linux 에서 동작한다. 금액은 원 단위 정수, 날짜는 `YYYY-MM-DD HH:MM:SS` 문자열, CSV 는 엑셀 호환 UTF-8 BOM 으로 저장한다. 급여는 2026 년 최저시급(10,320 원)과 근로기준법의 주휴수당(주 15 시간 이상)·연장수당(1.5 배) 규칙을 따른다.

③ **신뢰성/안정성/내구성** — 결제·환불·테이블 분할처럼 여러 테이블을 바꾸는 작업은 하나의 트랜잭션으로 묶고, 중간에 실패하면 되돌린다(rollback). 업무 규칙 위반은 `PosError` 로 구분해 안내 창만 띄우고, 예상하지 못한 오류는 `data/error.log` 에 기록한 뒤 프로그램을 계속 실행한다. 백업·복원과 자동 업그레이드로 데이터 손실을 막는다.

④ **경제성·미학** — 무료 도구만 사용하고 일반 PC 한 대로 운영할 수 있다. 화면은 테이블 상태 색(빈 테이블/이용 중/결제 중), 품절 회색 처리, 미전송 파란색처럼 상태를 색으로 구분했다.

> ***AI 활용*** *— 프롬프트 "일단 1~4 까지 하는거랑 6~8 까지 진행 하면 될거 같은데 5 는 너무 확률로 인해서 불안정할거 같아"에 대해 AI 가 계좌이체·옵션·인쇄·부분 환불·기간 비교·복원을 구현했고, 발주 추천은 결과가 불안정하다는 판단에 따라 제외했다. 위 표의 대안 비교는 각 단계에서 AI 와 논의한 내용을 정리한 것이다.*

5. 자료구조 설계

프로그램의 모든 상태는 SQLite 테이블 25 개에 저장한다. 화면은 필요할 때마다 DB 에서 다시 읽으므로 프로그램을 껐다 켜도 진행 중인 주문이 그대로 남는다.

5.1 테이블 구성

영역	테이블	내용
메뉴·재고	categories, menus, menu_options, ingredients, recipes, stock_logs	분류, 메뉴(가격·원가·할인율), 옵션, 재료 재고, 레시피, 재고 변동 이력
주문	tables_, orders, order_items	테이블, 주문(상태·금액 스냅샷), 주문 항목
결제	payments, refunds	결제 1 건씩(분할 결제), 환불 1 건씩(전체·부분)
시재	cash_sessions, cash_movements, cash_checks	시재 개시·정산, 입출금, 점검 기록(권종별 수량)
회원·할인	members, point_logs, coupons, time_discounts	회원·포인트 이력, 쿠폰, 시간대 할인
직원	staff, work_logs	직원(시급/월급), 출퇴근 기록
예약	reservations, reservation_items	예약과 예약 메뉴
운영	settings, closings, audit_logs	설정(비밀번호 해시·프린터), 마감 보고서, 작업 로그

5.2 주문과 주문 항목: orders / order_items

주문은 결제가 시작되면 금액을 잠그고(`locked`), 그 시점의 금액 계산 결과를 저장한다. 이후에는 메뉴 가격이나 할인 설정이 바뀌어도 주문 금액이 변하지 않는다.

멤버	자료형	의미
status	TEXT	OPEN(진행) / PAID(결제 완료) / CANCELLED(취소) / REFUNDED(전체 환불)
table_id, order_type	INTEGER, TEXT	테이블(포장이면 NULL), DINE_IN / TAKEOUT
created_at, completed_at, paid_at	TEXT	접수·조리 완료·결제 완료 시각
locked	INTEGER	1 이면 결제 시작됨 → 메뉴·할인 변경 불가
subtotal ~ total	INTEGER	할인 전 금액, 메뉴/시간대/쿠폰 할인, 사용 포인트, 최종 금액(잠금 시 저장)
refunded_amount	INTEGER	지금까지 환불한 금액(부분 환불 누적)

order_items 멤버	자료형	의미
name, unit_price, unit_cost, discount_rate	TEXT, INTEGER	주문 시점의 메뉴명·단가(옵션 포함)·원가·할인율
option_ids, options, memo	TEXT	선택 옵션 id 와 표시 문자열, 요청사항
qty, paid_qty, refunded_qty	INTEGER	수량, 메뉴별 분할로 결제한 수량, 환불한 수량
ticket_qty	INTEGER	주방 주문서로 이미 보낸 수량(차이만 다음 주문서에 출력)
status, ordered_at, done_at	TEXT	WAITING/COOKING/DONE, 조리 시간 계산용 시각

주문 상태는 다음처럼만 바뀐다.

```
OPEN ──(남은 금액 0 원)──▶ PAID ──(전체 환불)──▶ REFUNDED
  │                          └─(부분 환불: 상태 유지, refunded_amount 증가)
  ├──(결제 전 취소)──────────▶ CANCELLED
  └──(분할 결제 중 결제 취소)─▶ CANCELLED
```

5.3 결제·환불·시재: payments / refunds / cash_sessions

테이블	주요 멤버	의미
payments	method, amount, received, change, cash_session_id	결제 1 건. 현금은 받은 돈·거스름돈과 그때 열린 시재 id
refunds	method, amount, partial, detail, reason, cash_session_id	환불 1 건. 결제수단별로 나눠 저장, 부분 환불 여부

테이블	주요 멤버	의미
cash_sessions	opening_cash, expected_cash, counted_cash, difference, status	시재 1 회(개시~정산). 정산 시 예상·실제·과부족 저장
cash_checks	kind(CHECK/CLOSE), denominations(JSON)	중간 점검·정산 때 센 권종별 수량

결제와 환불을 각각 한 줄씩 쌓는 방식이라, 결제수단별 매출은 `결제 합계 - 환불 합계`로 계산되고 기록은 지워지지 않는다. 권종별 수량은 개수가 고정된 작은 데이터라 별도 테이블 대신 JSON 문자열(`{"50000": 2, "1000": 7}`)로 저장했다.

> ***AI 활용*** *— 프롬프트 "시재점검을 위한 비즈니스 로직 추가해야 할것 같음"에 대해 AI 가 시재 3 개 테이블과, 결제·환불 행에 시재 id 를 남기는 방식을 제안했다.*

6. 모듈 설계

업무 로직은 14 개 모듈로 나뉘며 모두 `Service`를 상속해 같은 DB 연결을 쓴다. 규칙 위반은 `PosError`를 던지고, 화면은 이를 받아 안내 창으로 보여준다.

6.1 menu — 메뉴·옵션·재고

함수	입력	출력	역할
resolve_options	메뉴, 옵션 id 목록	옵션, 추가금, 표시문	같은 그룹 중복 선택 거부, 추가금 합산
available_qty	메뉴, 옵션	만들 수 있는 수량	재료별 재고 ÷ 사용량의 최솟값
consume	메뉴, 수량, 옵션	재고 경고 목록	재료 차감(음수면 복구), 재고 이력 기록
adjust_stock	재료, 증감	없음	입고·폐기. 재고가 음수가 되면 거부

6.2 order — 주문

함수	역할
create_order	테이블에 진행 주문이 있으면 그것을 반환, 없으면 생성. 일별 주문번호
add_item	옵션·요청사항 검증, 재고 차감, 같은 조건이면 수량 합산
set_item_qty / cancel_order	수량 변경·취소. 조리 전 항목만 재고 복구
merge_tables / split_items	테이블 이동·합치기, 메뉴·수량 골라 나누기(트랜잭션)
kitchen_ticket	`qty − ticket_qty` 차이만 주방 주문서로 만들고 전송 표시
complete_item	조리 완료 시각 기록, 모든 항목 완료 시 주문 완료 시각

6.3 discount · payment — 금액 계산과 결제·환불

함수	입력	출력	역할
calculate	주문	금액 내역	메뉴→시간대→쿠폰→포인트 순서 계산. 잠긴 주문은 저장값 사용
pay	주문, 수단, 금액/메뉴	결제 결과	현금은 시재·거스름돈 확인, 첫 결제 시 잠금, 0 원이면 완료
split_amount_for_items	주문, 메뉴별 수량	결제 금액	주문 할인을 비율로 나눈 금액, 10 원 반올림
refund_items	주문, 메뉴별 수량, 수단	환불액	부분 환불. 수단별 배분, 포인트 비율 회수
refund	주문, 사유	환불액	남은 금액 전체 환불 또는 분할 결제 취소

6.4 cash — 시재

함수	역할
open / current / require_open	시재 개시(한 번에 하나), 현금 거래 전 개시 여부 확인
add_movement	입금·출금. 사유 필수, 금전함보다 많이 출금 불가
summary	예상 시재 = 개시 + 현금 매출 − 현금 환불 + 입금 − 출금
check / close	권종별 수량으로 과부족 기록. 정산은 사유·관리자 규칙 적용 후 시재 닫기
report_text / day_summary	시재 정산서, 일일 마감 보고서용 요약

6.5 그 밖의 모듈

모듈	역할
member	회원 등록·검색, 등급(일반/실버/골드), 포인트 증감·사용 규칙
staff	직원(최저임금 검사), 출퇴근, 근무 기록 중복 검사, 월 급여 계산
reservation	예약 등록(30 분 이후, 테이블 중복 검사), 알림 대상, 주문 전환
stats	요약·메뉴·시간대·유형·결제수단·추이·처리 시간·기간 비교
report	영수증, 일일 마감, CSV, 백업·복원(무결성 검사)
printer	OS 기본 프린터 인쇄, 꺼짐·실패 시 `prints/` 파일 저장
auth	관리자 로그인(해시 비교, 3 회 실패 잠금), 비밀번호 변경, 작업 로그
table	테이블 추가·수정·삭제(진행 주문이 있으면 삭제 불가)

모듈	역할
db / system	스키마·초기 데이터·자동 업그레이드 / 서비스 생성과 연결 공유
gui (10 개)	app(탭·모드·예약 알림), order_view, payment_dialog, kitchen, cash_view, stats_view, admin_views, member_view, reservation_view, widgets(공용 위젯·막대 그래프)

화면 모듈을 분리했기 때문에 GUI 없이 `PosSystem(":memory:")`만 만들어 모든 업무 규칙을 테스트할 수 있다.

> ***AI 활용*** *— 프롬프트 "이거 exe 프로그램으로 진행"에 대해 AI 가 PyInstaller 빌드 설정을 만들고, exe 에서 데이터 경로·콘솔 출력·오류 로그가 동작하도록* `main.py` *를 고쳤다. Wine 환경에서 Tk 가 실행되지 않는 원인(패키지 설치 도구가* `_tkinter.pyd` *파일을 손상시킨 문제)도 이 과정에서 찾아 해결했다.*

7. 주요 알고리즘

핵심 알고리즘은 여섯 가지이며, 모두 금액을 원 단위 정수로 계산해 반올림 오차가 쌓이지 않게 했다.

7.1 주문 금액 계산 (calculate)

1. 할인 전 금액 = Σ(단가 × 수량). 단가에는 옵션 추가금이 포함된다.
2. 메뉴 할인 = Σ(⌊단가 × 할인율 ÷ 10⌋ × 10 × 수량) — 10 원 미만 절사
3. 시간대 할인 = ⌊(1 단계 후 금액) × 할인율 ÷ 10⌋ × 10. 주문 접수 시각·요일에 맞는 할인 중 가장 큰 1 개
4. 쿠폰 = 정률이면 비율, 정액이면 금액(남은 금액 이내). 최소 주문 금액·만료일·사용 횟수 검사
5. 포인트 = min(사용 포인트, 남은 금액). 최종 금액 = 앞 단계를 모두 뺀 금액

예: 월요일 15:30, 참치김밥(10% 할인) 2 개 + 떡볶이 1 개 + 신규 쿠폰 1,000 원 → `13,500 − 900 − 1,260 − 1,000 = 10,340` 원 (단위 테스트로 검증).

7.2 재고 차감과 자동 품절

메뉴 1 개(+옵션)에 필요한 재료량 `need[재료]`를 구한 뒤, 주문 가능 수량은 다음과 같다. 0 이면 메뉴 버튼을 비활성화하고, 입고로 재고가 늘면 자동으로 다시 활성화된다.

```
주문 가능 수량 = min over 재료 i ( ⌊ 재고_i ÷ need_i ⌋ )
```

7.3 분할 결제·부분 환불 금액 배분

주문 단위 할인(시간대·쿠폰·포인트)을 메뉴에 비율대로 나눈다. 선택한 메뉴로 주문이 끝나면(마지막 결제·전체 환불) 계산 대신 남은 금액을 그대로 써서 끝전이 남지 않게 한다.

```
비율 r = 최종 결제 금액 ÷ (할인 전 금액 − 메뉴 할인)
선택 금액 = Σ (단가 − 메뉴 할인) × 선택 수량
결제(환불) 금액 = min( 남은 금액, round(선택 금액 × r ÷ 10) × 10 )
```

부분 환불은 이렇게 구한 금액을 결제수단별로 나눈다. 고른 수단부터 그 수단으로 낸 금액만큼 환불하고, 부족하면 남은 금액이 큰 수단 순으로 채운다. 그래서 어떤 수단도 낸 돈보다 많이 환불되지 않는다.

7.4 급여 계산 (payroll)

1. 근무 기록을 날짜별(`daily`)·ISO 주별(`weekly`) 근무시간으로 모은다.
2. 아르바이트: `시급 × 근무시간` + 주휴수당(주 15 시간 이상 주마다 `min(주 근무시간, 40) ÷ 40 × 8 × 시급`) + 하루 8 시간 초과분 0.5 배 가산
3. 정직원: `월급` + 연장수당(하루 8 시간 초과분 × 통상시급(월급 ÷ 209) × 1.5)

예: 시급 10,400 원, 월~금 하루 4 시간(주 20 시간) → 기본급 208,000 원 + 주휴수당 `20 ÷ 40 × 8 × 10,400 = 41,600` 원.

7.5 시재 예상 금액과 과부족

```
예상 시재 = 개시 금액 + Σ현금 결제 − Σ현금 환불 + Σ입금 − Σ출금
실제 시재 = Σ (권종 × 수량)          권종 = 50,000 · 10,000 · 5,000 · 1,000 · 500 · 100 · 50 · 10
과부족   = 실제 − 예상          (≠0 이면 사유 필수, |과부족| ≥ 10,000 이면 관리자만 정산)
```

현금 결제 금액은 `받은 돈 − 거스름돈`으로 저장되므로 거스름돈을 따로 빼지 않아도 된다. 각 현금 결제·환불에 시재 id 를 남겨, 개시·정산 시각 경계에서도 오차가 없다.

7.6 기간 비교의 비교 기간

선택 기간	비교 기간	예
한 달 전체	전월 전체	10/1~10/31 → 9/1~9/30
한 해 전체	전년 전체	2026 년 → 2025 년
월 초 ~ 오늘	지난달 같은 날짜	10/1~10/14 → 9/1~9/14
그 외	같은 길이의 바로 전 기간	10/12~10/18 → 10/5~10/11

7.7 시간 복잡도

작업	복잡도	비고
메뉴 추가·금액 계산	O(n)	n = 주문 항목 수(최대 50)
주문 가능 수량	O(r)	r = 레시피 재료 수
분할·부분 환불 금액	O(n)	항목마다 상수 시간
수단별 환불 배분	O(m log m)	m = 결제수단 수(최대 4)
통계 집계	O(N) (SQL GROUP BY)	N = 기간 내 주문 항목 수, 인덱스 사용
급여 계산	O(직원 수 × 근무 기록 수)	월 단위

> ***AI 활용*** *— 프롬프트 "지금 제한사항이랑 확장은 다 완성 된건가?"에 대해 AI 가 구현 상태를 점검했고, 이어서 분할 결제·부분 환불의 금액 배분 규칙과 시재 계산식을 단위 테스트의 예시 값으로 검증했다.*

8. 개발 내용(코드) 및 실행 화면

전체 코드는 Python 3 로 작성한 7,030 줄(테스트 678 줄 별도)이며, 아래는 각 기능의 핵심 부분이다. 전체 소스는 제출 파일 `kumoh_pos` 에 있다.

8.1 파일 구성

파일	줄 수	내용
main.py	85	실행 진입점, exe 데이터 경로·오류 로그
pos/config.py · db.py · system.py	69 · 476 · 45	제한사항 상수 / 스키마·마이그레이션·초기 데이터 / 서비스 묶음
services/order.py	393	주문, 옵션·요청사항, 테이블 병합·분할, 주방 주문서
services/payment.py	369	결제·분할 결제·전체/부분 환불
services/menu.py	313	메뉴·옵션·레시피·재고
services/stats.py	275	통계·기간 비교
services/report.py	266	영수증·마감·CSV·백업/복원
services/cash.py	212	시재 개시·입출금·점검·정산
services/staff.py · discount.py	206 · 194	근태·급여 / 할인·금액 계산
services/기타 6 개	461	member, reservation, printer, auth, table, 공통
gui/ 10 개 모듈	3,524	화면(주문 612, 결제 434, 관리자 995 등)
pos/demo.py	140	샘플 데이터 생성기
tests/test_pos.py	678	단위 테스트 59 개

8.2 핵심 코드

① 주문 금액 계산 — 할인을 정해진 순서로 적용하고, 쿠폰 조건이 맞지 않으면 결제를 막지 않고 경고만 남긴다.

```
items = self.db.query("SELECT * FROM order_items WHERE order_id=?", (order_id,))
subtotal = sum(i["unit_price"] * i["qty"] for i in items)
menu_discount = sum(self.item_discount(i["unit_price"], i["discount_rate"]) * i["qty"]
                    for i in items)
amount = subtotal - menu_discount

time_discount = 0
created = datetime.strptime(order["created_at"], config.DATETIME_FMT)
td = self.active_time_discount(created)
if td and amount > 0:
    time_discount = floor_unit(amount * td["rate"] / 100, config.DISCOUNT_UNIT)
    result["time_discount_name"] = f"{td['name']}({td['rate']}%)"
amount -= time_discount
```

```
coupon_discount = 0
if order["coupon_code"]:
    try:
        coupon = self.check_coupon(order["coupon_code"], amount)
        coupon_discount = self.coupon_amount(coupon, amount)
        result["coupon_name"] = coupon["name"]
    except PosError as e:
        result["warnings"].append(f"쿠폰 미적용: {e}")
amount -= coupon_discount

points_used = min(order["points_used"], amount)
amount -= points_used
```

② **첫 결제 시 금액 잠금** — 금액을 저장하고 쿠폰 사용 횟수와 포인트를 이 시점에 한 번만 차감한다.

```
def _lock(self, order_id: int) -> None:
    """첫 결제 시 금액을 확정(스냅샷)하고 쿠폰·포인트를 차감한다."""
    order = self._order(order_id)
    calc = self.discount.calculate(order_id)
    coupon = order["coupon_code"] if calc["coupon_discount"] > 0 else None
    self.db.execute(
        "UPDATE orders SET locked=1, subtotal=?, menu_discount=?, time_discount=?, "
        "coupon_discount=?, points_used=?, total=?, coupon_code=? WHERE id=?",
        (calc["subtotal"], calc["menu_discount"], calc["time_discount"],
         calc["coupon_discount"], calc["points_used"], calc["total"], coupon, order_id),
    )
    if coupon:
        self.db.execute("UPDATE coupons SET used_count=used_count+1 WHERE code=?",
(coupon,))
    if calc["points_used"]:
        self.member.adjust_points(order["member_id"], -calc["points_used"], "포인트 사용",
                                  order_id)
```

③ **주문 가능 수량** — 옵션 재료까지 합친 필요량으로 재고를 나눈다.

```
def available_qty(self, menu_id: int, option_ids=()) -> int:
    """현재 재고로 만들 수 있는 최대 수량. 레시피가 없으면 무제한(9999)."""
    need = self._requirements(menu_id, option_ids)
    if not need:
        return 9999
    stock = {r["id"]: r["stock"] for r in self.db.query("SELECT id, stock FROM
ingredients")}
    return max(0, min(math.floor(stock.get(i, 0) / q + 1e-9) for i, q in need.items()))
```

④ **메뉴별 분할 결제 금액** — 주문 할인을 비율로 나누고, 마지막 결제는 남은 금액 전체로 끝전을 정리한다.

```
net_total = calc["subtotal"] - calc["menu_discount"]
selected = 0
all_paid_after = True
for item_id, item in items.items():
    qty = item_qtys.get(item_id, 0)
    if qty < 0 or qty > item["qty"] - item["paid_qty"]:
        raise PosError(f"'{item['name']}'의 결제 수량이 올바르지 않습니다.")
    unit_net = item["unit_price"] - self.discount.item_discount(
```

```
        item["unit_price"], item["discount_rate"])
    selected += unit_net * qty
    if item["paid_qty"] + qty < item["qty"]:
        all_paid_after = False
if not any(item_qtys.values()):
    raise PosError("결제할 메뉴를 선택하세요.")
if all_paid_after:
    return calc["remaining"]
ratio = calc["total"] / net_total if net_total else 0
# 10원 단위 반올림(마지막 결제가 끝전을 정산)
return min(calc["remaining"], round(selected * ratio / 10) * 10)
```

⑤ **부분 환불의 결제수단별 배분** — 계획을 먼저 세운 뒤 현금이 포함되면 시재 조건을 검사하고, 통과해야 DB에 쓴다.

```
plan, left = [], amount
for m in order_methods:
    take = min(left, avail[m])
    if take:
        plan.append((m, take))
        left -= take
cash_session = self._cash_refund_session(sum(t for m, t in plan if m == "CASH"))
```

⑥ **주방 주문서** — 주문 수량과 이미 보낸 수량의 차이만 출력한다(+는 추가, -는 취소).

```
def pending_ticket_lines(self, order_id: int) -> list[tuple]:
    """주방에 아직 전달하지 않은 변경분: (항목, 추가 수량(+) 또는 취소 수량(-))."""
    return [(it, it["qty"] - it["ticket_qty"]) for it in self.items(order_id)
            if it["qty"] != it["ticket_qty"]]
```

⑦ **급여 계산** — 아르바이트는 주휴수당과 연장 가산, 정직원은 연장수당을 더한다.

```
if staff["kind"] == "PART":
    base = int(total_hours * hourly)
    holiday = int(sum(
        min(h, 40) / 40 * 8 * hourly
        for h in weekly.values() if h >= config.WEEKLY_HOLIDAY_MIN_HOURS
    ))
    overtime_pay = int(overtime_hours * hourly * (config.OVERTIME_RATE - 1))
else:
    base = staff["monthly_salary"]
    holiday = 0
    overtime_pay = int(overtime_hours * hourly * config.OVERTIME_RATE)
```

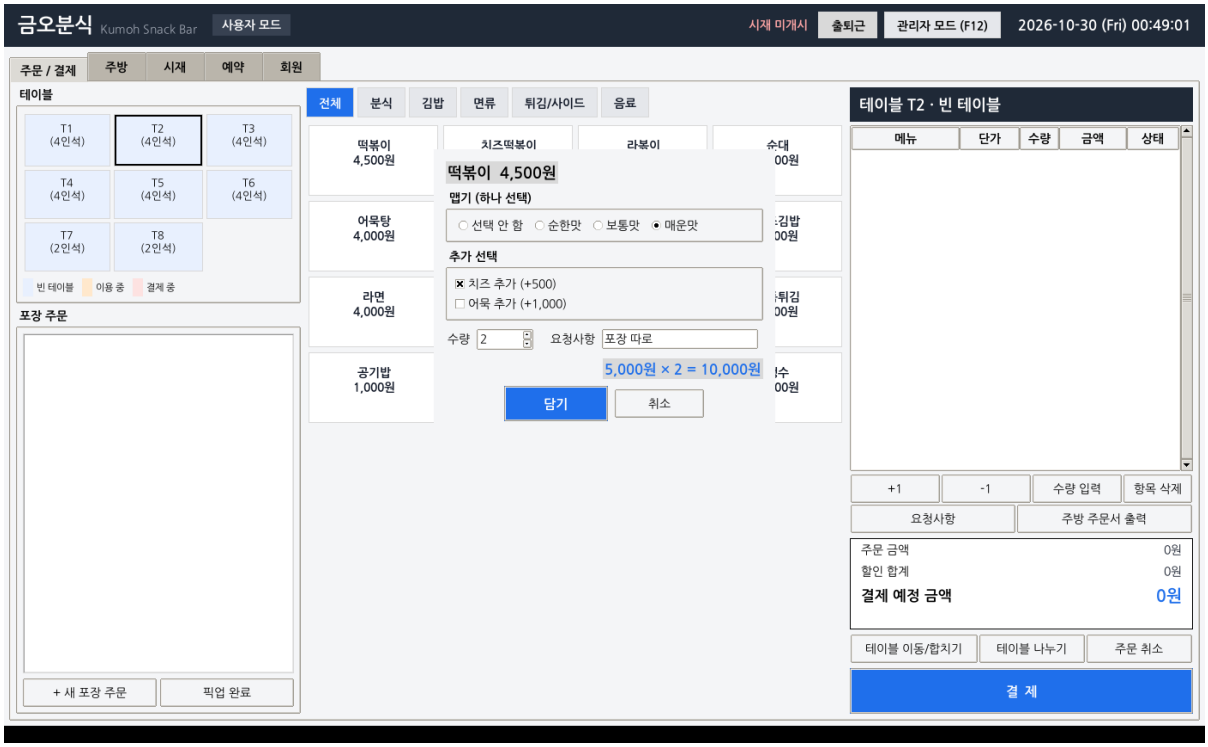
⑧ **시재 정산 규칙** — 과부족 사유와 관리자 기준을 정산 직전에 검사한다.

```
if kind == "CLOSE":
    if diff and not note:
        raise PosError(f"과부족 {diff:+,}원이 있습니다. 사유를 입력해야 정산할 수 있습니다.")
    if abs(diff) >= config.CASH_DIFF_ADMIN_LIMIT and self.db.mode != "ADMIN":
        raise PosError(f"과부족이 {config.CASH_DIFF_ADMIN_LIMIT:,}원 이상이면 관리자 모드에서만
"
                       "정산할 수 있습니다.")
```

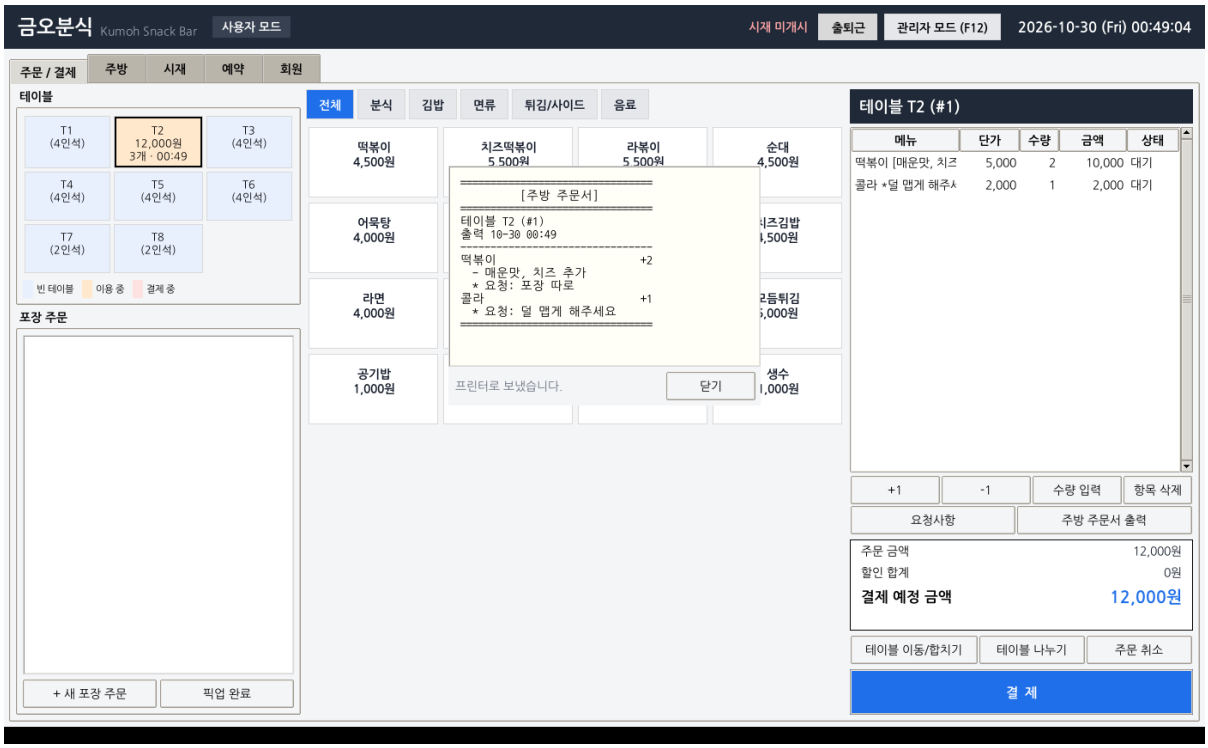
8.3 실행 화면

아래 화면은 프로그램을 실제로 실행해 샘플 데이터로 조작한 결과를 캡처한 것이다.

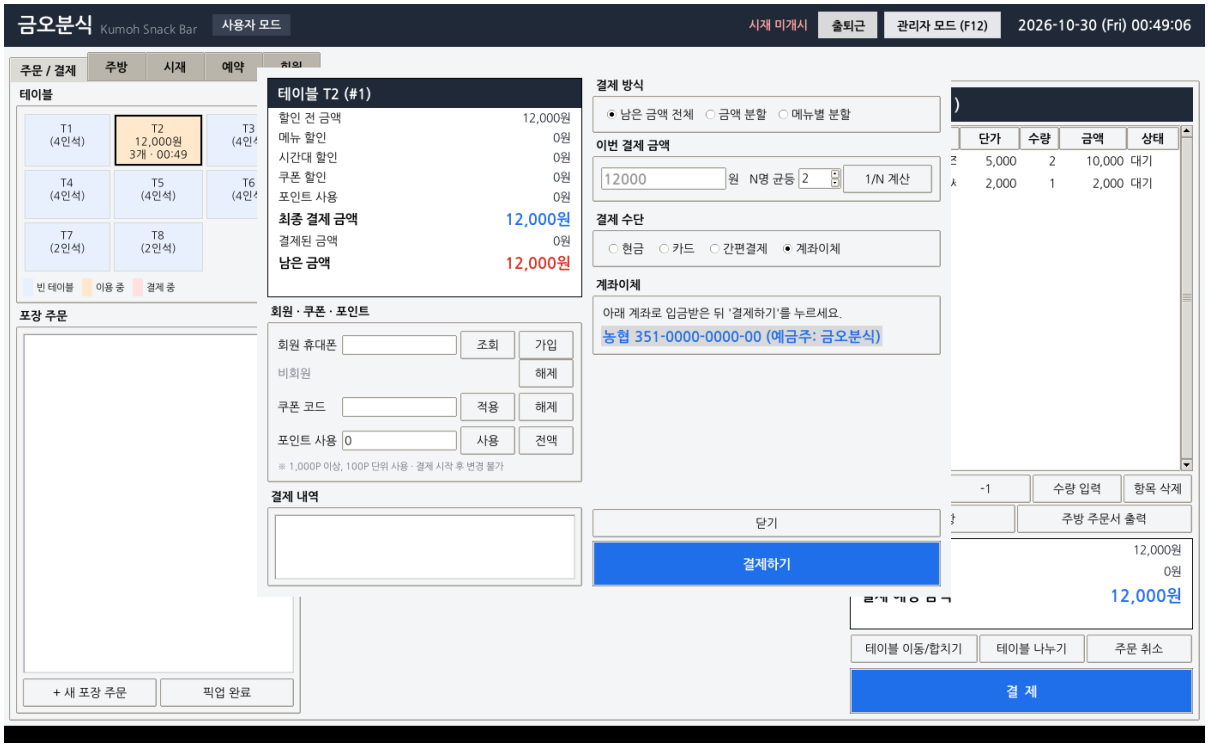
① **메뉴 옵션 선택** — 맵기는 하나만, 추가 선택은 여러 개 고를 수 있고 금액이 바로 계산된다.



② **주문 화면과 주방 주문서** — 왼쪽은 테이블 상태(색), 가운데 메뉴판, 오른쪽 현재 주문. 주문서에는 옵션과 요청사항이 함께 나간다.



③ **결제 창(계좌이체)** — 할인 단계별 금액, 회원·쿠폰·포인트, 결제 방식과 수단, 입금 계좌 안내.

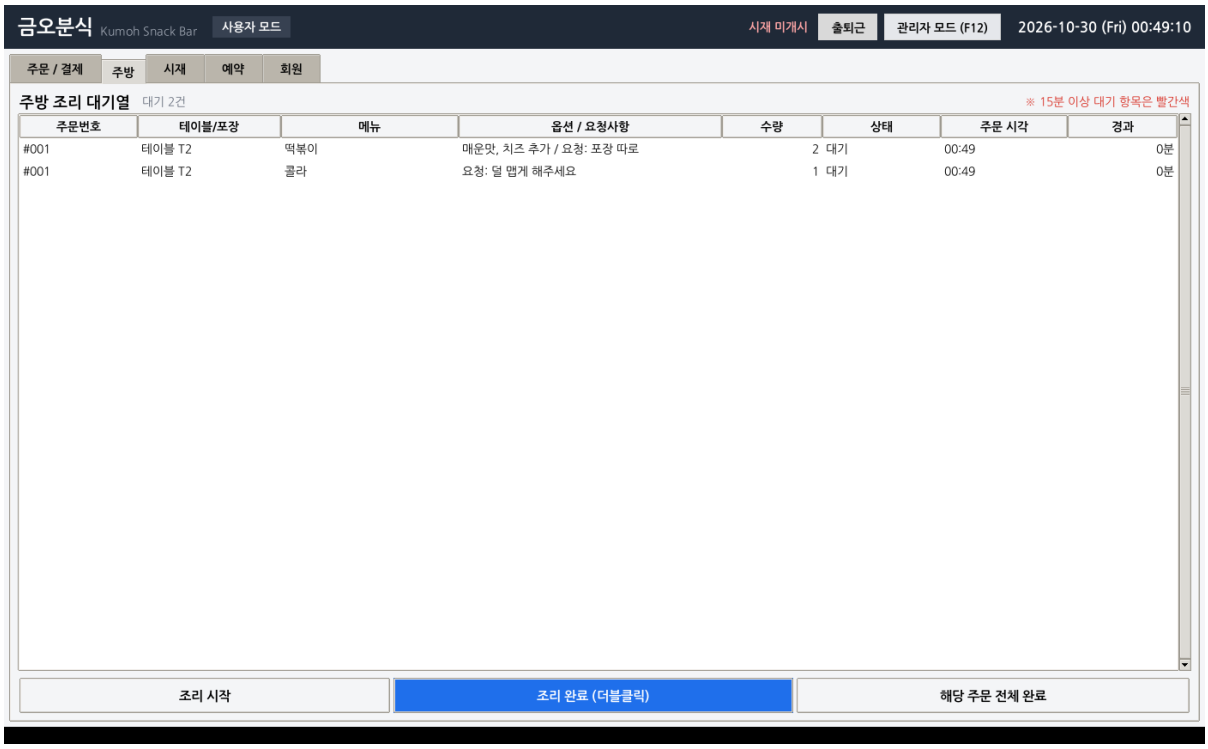


④ **상세 영수증** — 옵션, 메뉴 할인, 시간대 할인, 쿠폰, 카드+현금 분할 결제, 포인트 적립이 모두 표시된다.

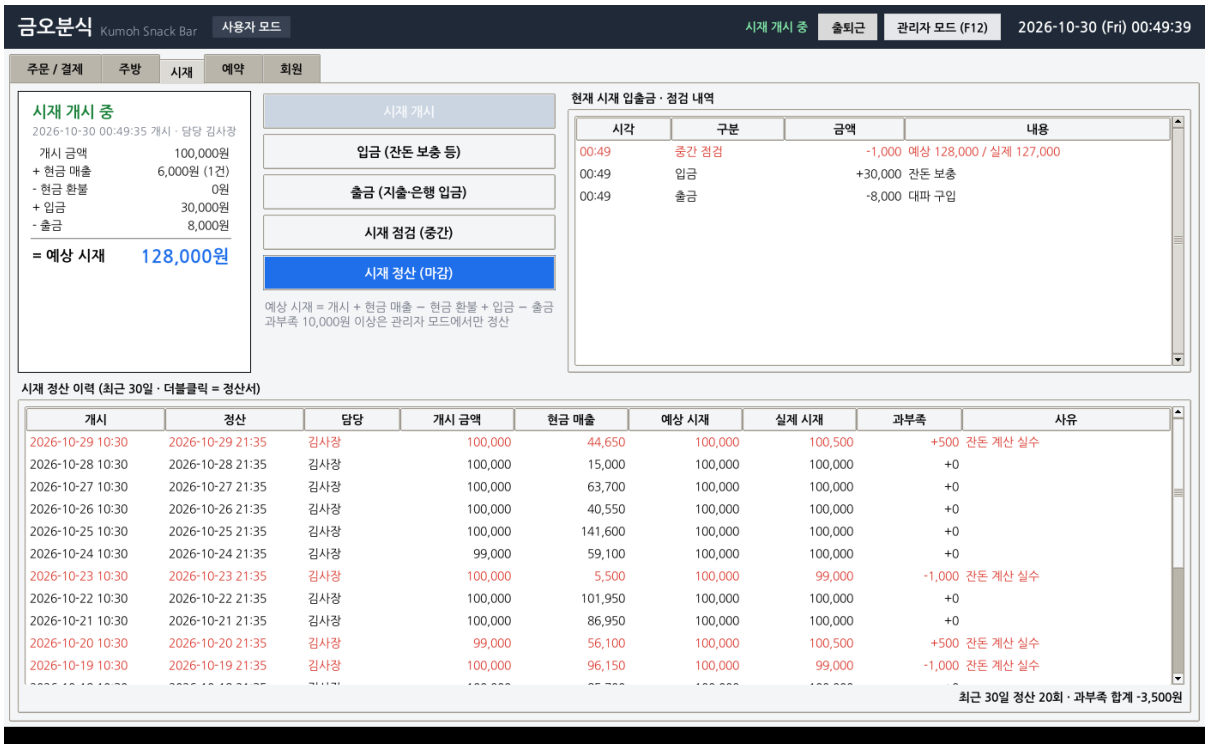
```
========================================
       금오분식 (Kumoh Snack Bar)
사업자 123-45-67890  TEL 054-478-0000
경북 구미시 대학로 61
========================================
주문번호 : 001  [매장] 테이블 T3
주문일시 : 2026-10-12 15:30:00
결제일시 : 2026-10-12 15:30:00
----------------------------------------
메뉴        단가  수량     금액
----------------------------------------
떡볶이       5,000    2       10,000
  + 매운맛, 치즈 추가
참치김밥      4,500    1        4,500
  └ 메뉴할인 10%                    -450
콜라        2,000    1        2,000
----------------------------------------
합계(할인 전)                    16,500 원
메뉴 할인              -450 원
시간대 할인 오후 한산시간 할인(10%) -1,600 원
쿠폰 할인(WELCOME1000)          -1,000 원
----------------------------------------
결제 금액            13,450 원
  (부가세 포함)                  1,223 원
----------------------------------------
카드               8,380 원
현금               5,070 원
  받은 금액          10,000 원
  거스름돈           4,930 원
----------------------------------------
```

```
회원 홍길동 (5678)                적립 134P
잔여 포인트                  134P
========================================
          이용해 주셔서 감사합니다!
========================================
```

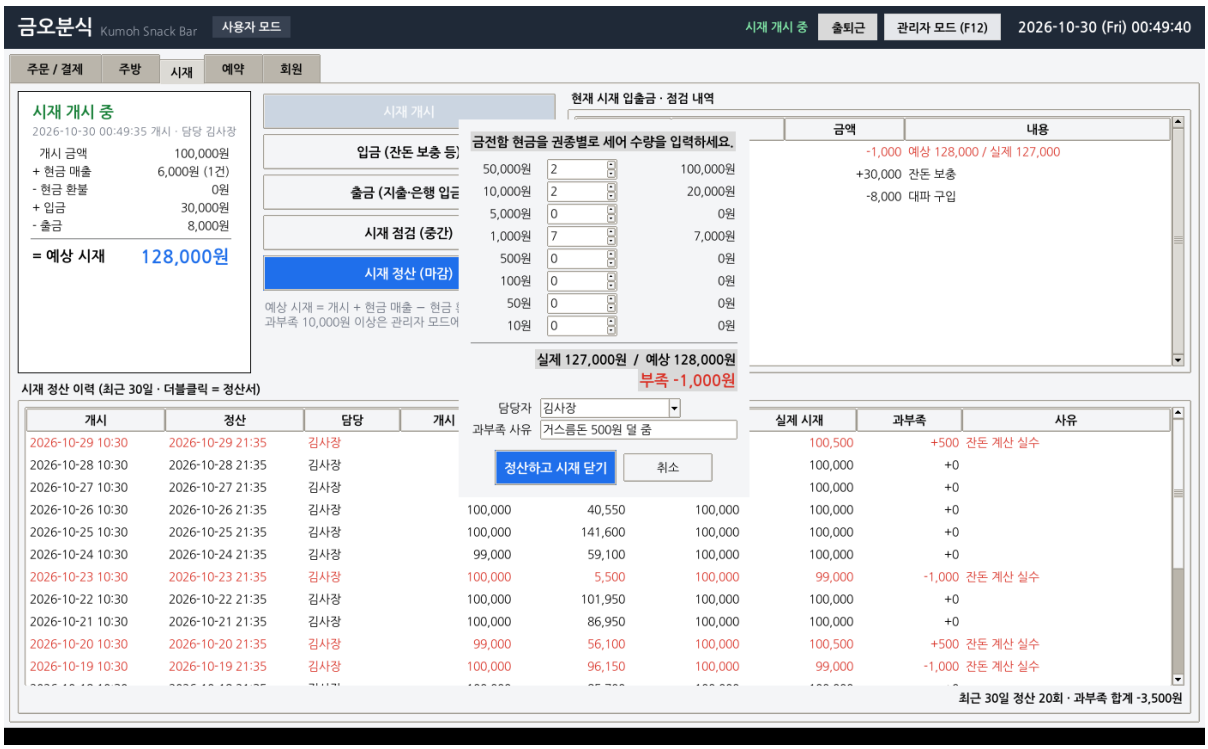
⑤ **주방 화면** — 조리 대기열에 옵션·요청사항과 경과 시간이 표시된다.



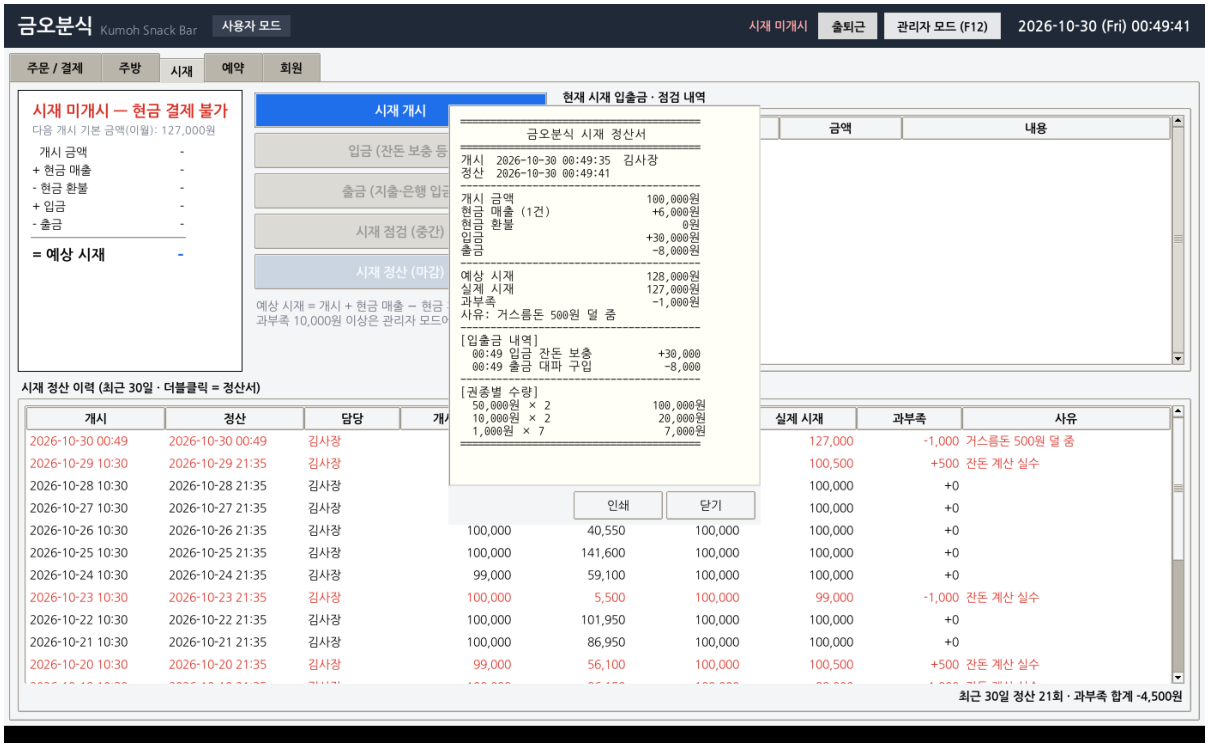
⑥ **시재 관리** — 예상 시재 계산 내역, 입출금·점검 기록, 최근 30 일 정산 이력과 과부족 합계.



⑦ **시재 정산** — 권종별 수량을 넣으면 실제·예상·과부족이 계산되고, 과부족 사유를 입력해야 정산된다.



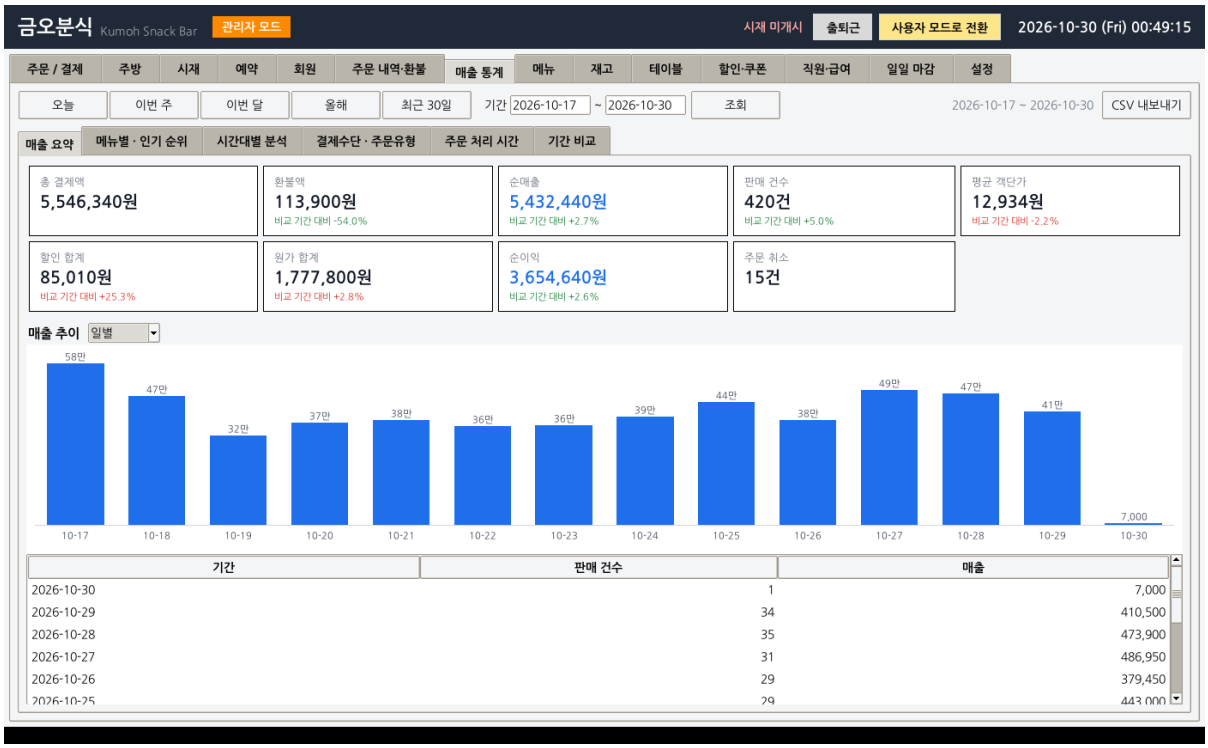
⑧ **시재 정산서**



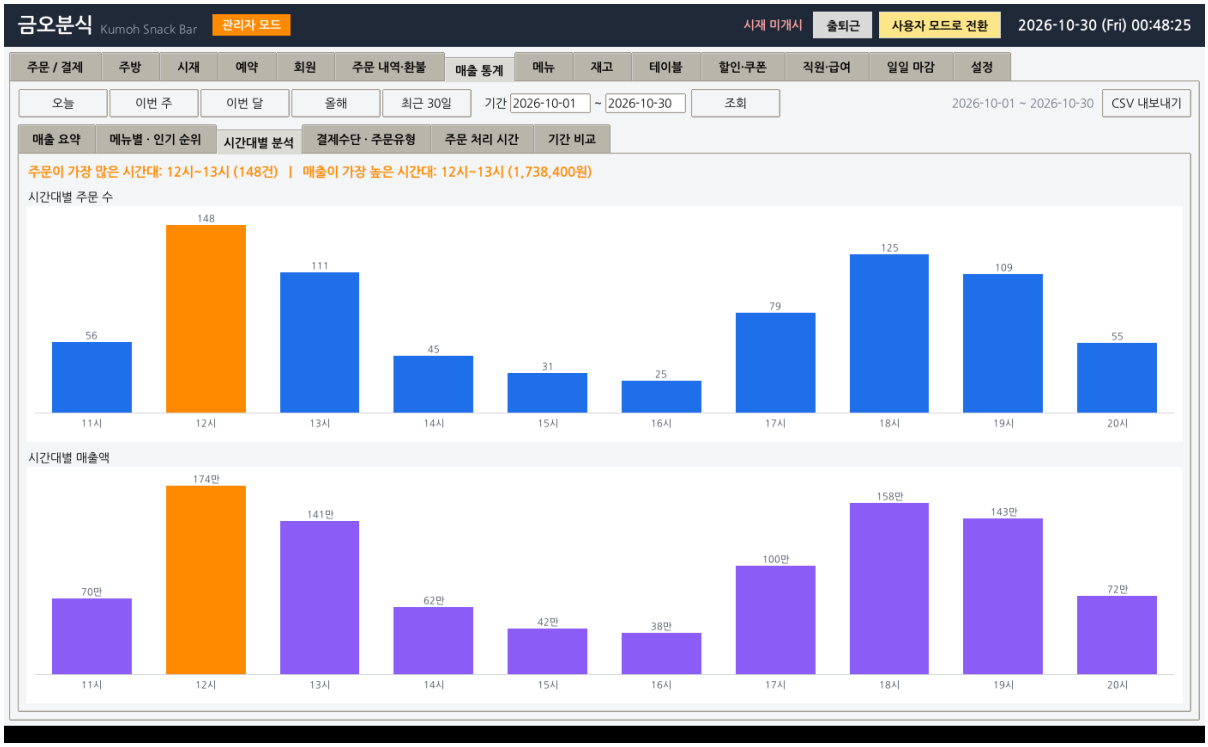
⑨ **부분 환불** — 메뉴·수량을 고르면 환불 금액이 계산되고, 환불 수단을 선택한다.



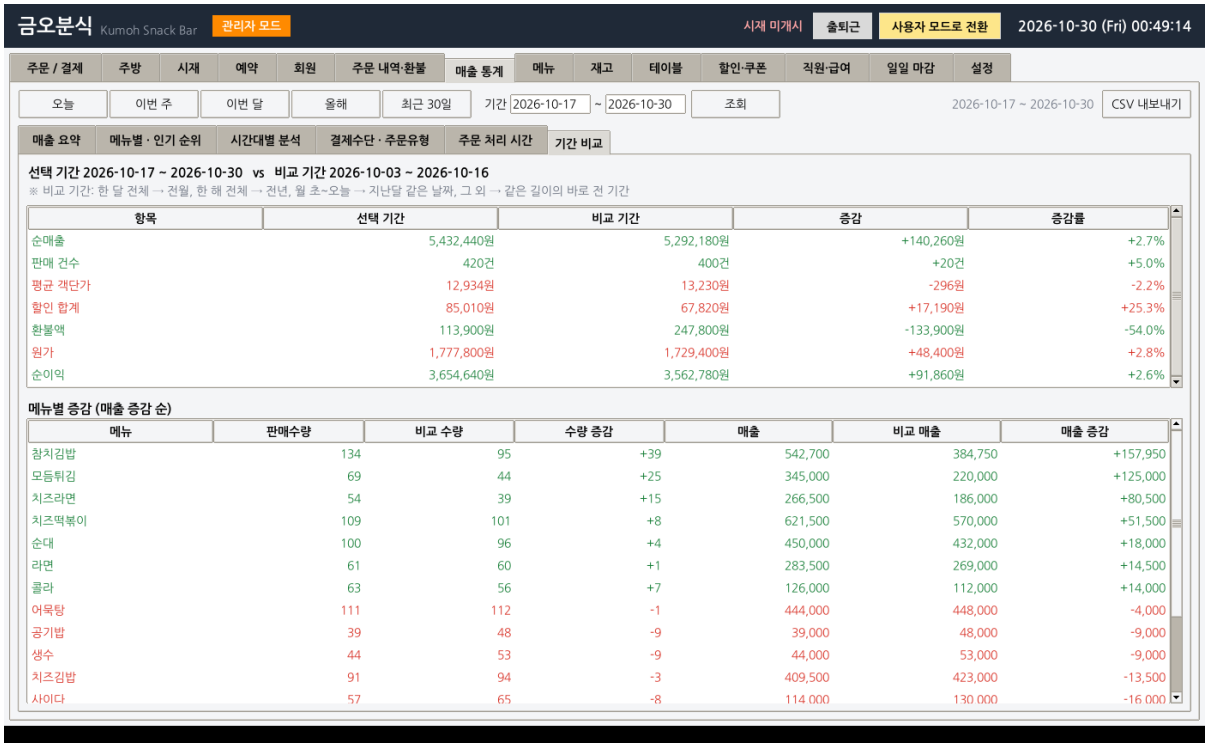
⑩ **매출 통계 요약** — 요약 카드에 비교 기간 대비 증감률이 표시된다(좋아지면 초록, 나빠지면 빨강).



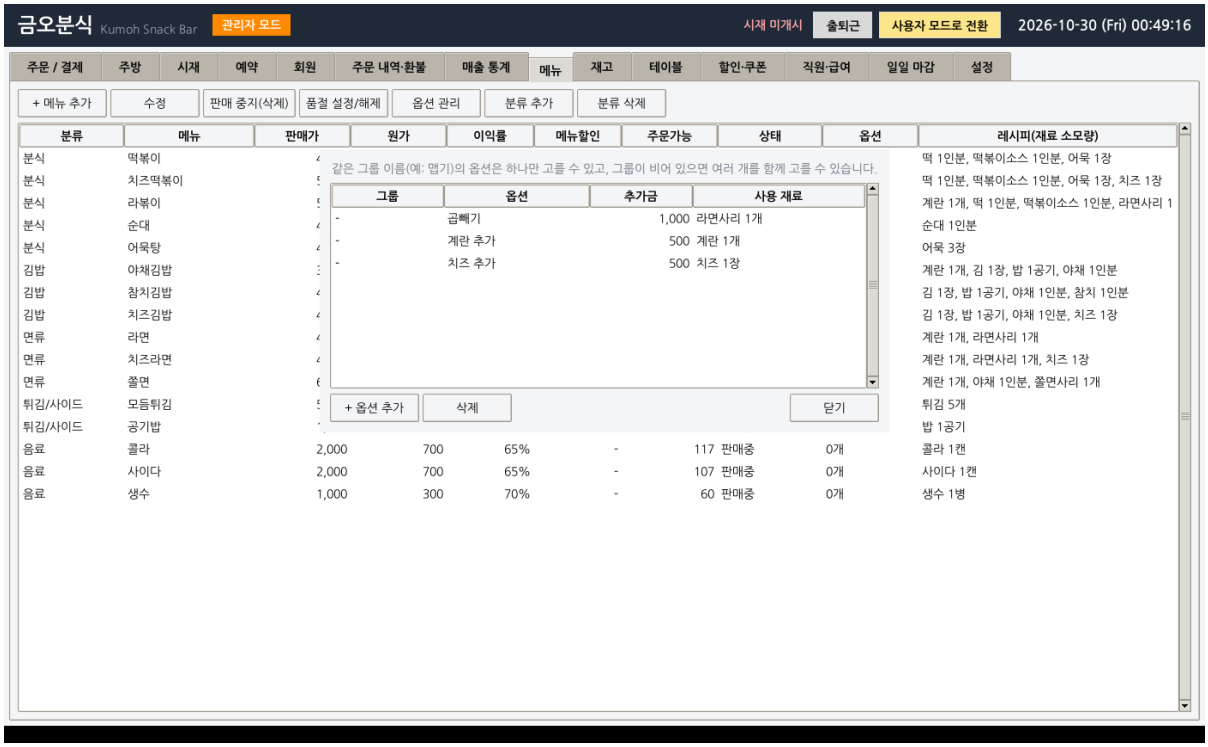
⑪ **시간대별 분석** — 주문이 가장 많은 시간대와 매출이 가장 높은 시간대를 주황색으로 강조한다.



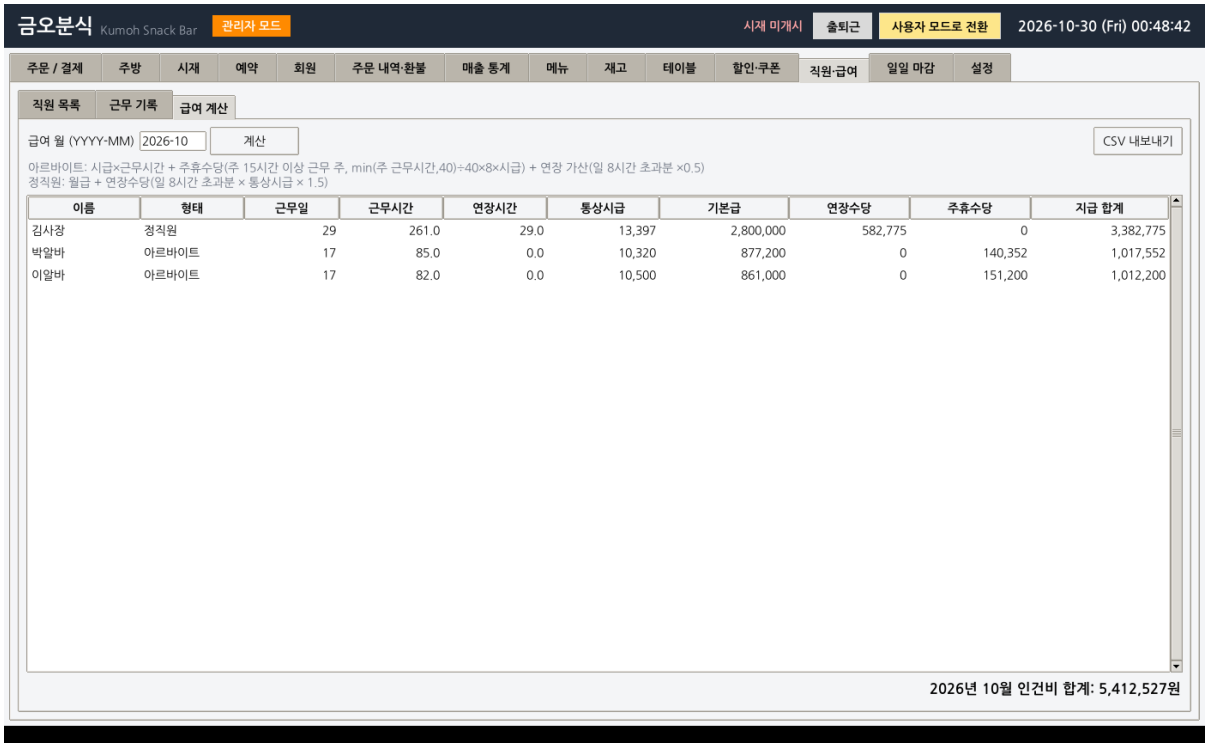
⑫ **기간 비교** — 선택 기간과 비교 기간의 지표·메뉴별 증감.



⑬ **메뉴 옵션 관리(관리자)** — 메뉴별로 옵션 그룹·추가금·사용 재료를 등록한다.



⑭ 급여 계산(관리자) — 근무일·근무시간·연장·주휴수당과 월 인건비 합계.

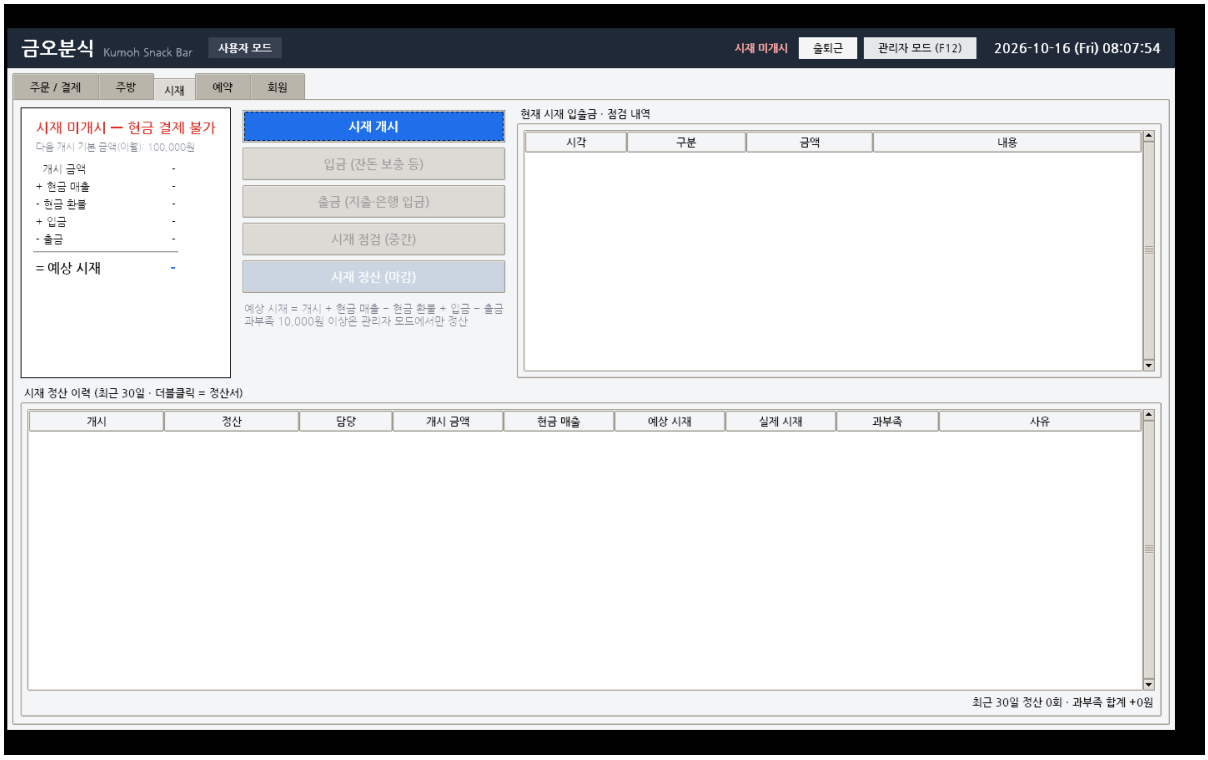


⑮ 일일 마감 보고서 — 샘플 데이터의 2026-10-29 마감 보고서. 결제수단별·유형별 매출, 취소/환불, 순이익, 시재 정산이 들어간다.

```
========================================
          금오분식 일일 마감 보고서
영업일 : 2026-10-29
출력  : 2026-10-30 00:53:45
========================================
```

```
총 주문 건수(접수)                    34 건
판매 완료 건수                        34 건
총 결제액                         434,390 원
할인 합계                          11,110 원
포인트 사용                            0 원
---------------------------------------
[결제수단별]
  현금 (7 건)                     119,750 원 (27.6%)
  카드 (16 건)                    200,800 원 (46.2%)
  간편결제 (9 건)                  93,740 원 (21.6%)
  계좌이체 (2 건)                  20,100 원 (4.6%)
---------------------------------------
[주문유형별]
  매장 (24 건)                      300,590 원
  포장 (10 건)                      133,800 원
---------------------------------------
[취소/환불]
  주문 취소 (0 건)                      0 원
  환불 (0 건)                           0 원
---------------------------------------
순매출 (결제-환불)                  434,390 원
원가 합계                         143,000 원
순이익 (판매액-원가)                291,390 원
---------------------------------------
[판매 TOP 5]
  1. 참치김밥                    17 개 68,850 원
  2. 라볶이                      13 개 75,500 원
  3. 생수                        11 개 11,000 원
  4. 라면                         9 개 40,000 원
  5. 치즈라면                     8 개 38,500 원
---------------------------------------
[시재 정산]
  10:30~21:35 김사장
    개시 금액                     100,000 원
    현금 매출                    +119,750 원
    입금 / 출금                  +0 / -119,750
    예상 / 실제                  100,000 / 99,000
    과부족                        -1,000 원
  과부족 합계                     -1,000 원
=======================================
```

⑯ **Windows exe 실행** — 이전 버전으로 만든 DB 를 그대로 둔 채 새 exe 를 실행한 화면. 시재 테이블이 자동으로 추가되고 시재 미개시 상태로 시작한다.



9. 실험 결과

단위 테스트 59 개를 모두 통과했고, GUI 자동 조작 시나리오 4 종과 exe 실행 테스트에서 오류가 없었다. 단위 테스트는 시간을 고정한 가짜 시계와 메모리 DB 로 실행해 언제 돌려도 같은 결과가 나온다.

9.1 기본 기능 테스트

구분	입력	기대 결과	결과
현금 거스름돈	떡볶이 4,500 원, 받은 금액 10,000 원	거스름돈 5,500 원, PAID	통과
할인 순서	월 15:30, 참치김밥(10%)×2+ 떡볶이, 쿠폰 1,000 원	13,500−900−1,260−1,000=10,340 원	통과
시간대 할인 요일	토요일 15:30 주문	평일 전용 할인 미적용	통과
재고 차감·복구	떡볶이 3 개 → 1 개로 변경 → 취소	떡 −3 → −1 → 원래대로	통과
자동 품절·재입고	순대 재고 2, 2 개 주문 후 입고 10	품절 → 판매 재개	통과
테이블 합치기·나누기	T1+T2 합치기, 떡볶이 3 개 중 1 개를 T4 로	항목 이동, 수량 분리	통과
금액 분할	9,000 원을 4,000 + 나머지	남은 금액 5,000 원 후 완료	통과

구분	입력	기대 결과	결과
메뉴별 분할	쫄면 / 떡볶이 따로 결제	6,000 원, 4,500 원	통과
포인트	30,000 원 결제 후 1,200P 사용	적립 300P(1%), 결제 3,300 원	통과
메뉴 옵션	떡볶이 2 개 + 매운맛 + 치즈 추가	단가 5,000 원, 치즈 재고 −2	통과
주방 주문서	떡볶이 2 출력 → 콜라 추가, 떡볶이 1 로 변경	두 번째 주문서에 콜라 +1, 떡볶이 취소 1 만	통과
부분 환불	16,500 원 결제, 쫄면 1 개 환불	환불 6,000 원, 순매출 10,500 원, 포인트 비율 회수	통과
수단별 환불 배분	현금 2,000 + 카드 10,000, 6,000 원 현금 지정	현금 2,000 + 카드 4,000	통과
기간 비교	10/12~18 vs 10/5~11	순매출 9,000 vs 4,500, +100%	통과
급여(주휴수당)	시급 10,400 원, 주 20 시간	기본 208,000 + 주휴 41,600	통과
급여(연장)	월급 4,180,000 원, 하루 10 시간	연장 2 시간 × 20,000 × 1.5 = 60,000	통과
시재 예상	개시 10 만, 현금 1.2 만, 환불 0.6 만, 입금 3 만, 출금 0.5 만	예상 131,000 원	통과
일일 마감	결제·시재 정산 후 마감	보고서 저장, 시재 정산 포함, 재마감 거부	통과
영수증 종류	상세 / 간이	간이에는 메뉴명 없음	통과
DB 복원	백업 → 회원 등록 → 복원	등록한 회원이 사라짐, 복원 전 백업 생성	통과

9.2 예외 처리 테스트

구분	입력	결과
수량 범위	한 메뉴 100 개	거부(1~99)
수동 품절	품절 설정한 라면 주문	거부
결제 중 메뉴 변경	일부 결제 후 메뉴 추가	거부(금액 잠금)
현금 부족	4,500 원에 4,000 원 받음	거부, 주문은 잠기지 않음 (rollback)
쿠폰 조건	최소 금액 미달, 만료 쿠폰	거부
포인트 규칙	500P 사용	거부(1,000P 이상)
옵션 그룹	순한맛 + 매운맛 동시 선택	거부(하나만)

구분	입력	결과
옵션 재료 부족	치즈 0 장에서 치즈 추가	옵션만 거부, 기본 메뉴는 주문 가능
요청사항	41 자 / 조리 시작 후 수정	거부
중복 환불	이미 환불한 수량 다시 환불	거부
시재 미개시	현금 결제, 현금 환불	거부(카드는 가능)
시재 출금	금전함보다 큰 출금, 사유 없음	거부
시재 정산	과부족 사유 없음 / 2 만원 차이(사용자 모드)	거부 / 관리자만 가능
권종	3,000 원권, 음수 수량	거부
최저임금	시급 9,000 원	거부
근무 기록	시간이 겹치는 기록	거부
예약	20 분 후 예약	거부(30 분 이후)
로그인	비밀번호 3 회 오류	30 초 잠금(맞는 비밀번호도 거부)
마감	미결제 주문 있음 / 시재 미정산	거부
복원	DB 가 아닌 파일	거부, 현재 데이터 유지
이전 버전 DB	옛 구조의 pos.db	새 컬럼 자동 추가 후 정상 실행

9.3 대량 데이터 성능

샘플 데이터 생성기로 90 일치 영업 데이터를 만들고 통계 7 종(요약, 메뉴, 시간대, 결제수단, 추이, 기간 비교, 처리 시간)을 계산하는 시간을 쟀다.

항목	값
생성 데이터	주문 2,856 건, 주문 항목 6,832 개, 결제 2,765 건, 시재 90 회
생성 시간	3.3 초
통계 7 종 계산 시간	0.18 초(5 회 평균)
결제수단 비율	카드 51.7% · 간편결제 23.6% · 현금 16.4% · 계좌이체 8.2%
매장 / 포장	1,920 건(71.3%) / 778 건(28.7%)
평균 주문 처리 시간	17.8 분
피크 시간대	12 시(주문 수·매출 모두)

90 일치 순매출(결제 총액 – 환불, 환불은 전체 67 건·부분 38 건)은 35,473,160 원으로, 주문별 실결제액의 합 및 결제수단별 금액의 합과 1 원 단위까지 일치했다. 시재 90 회는 모두 정산되었고 과부족 합계는 −18,500 원(샘플 생성기가 일부러 넣은 오차)이었다.

9.4 실행 환경 테스트

구분	방법	결과
GUI 전체 흐름	가상 화면(Xvfb)에서 화면을 자동 조작하는 시나리오 4 종	오류 0 건
시재 시나리오	미개시 현금 결제 → 개시 → 결제 → 입출금 → 점검 → 사유 없는 정산 → 정산 → 마감	규칙대로 거부·통과
exe 실행	Wine(리눅스의 Windows 호환 환경)에서 exe 실행, 주문·옵션·결제·영수증	정상 동작
exe 업그레이드	이전 버전 exe 가 만든 DB 로 새 exe 실행	시재·옵션 테이블 자동 추가
인쇄 실패	프린터 없는 환경에서 주방 주문서 출력	파일로 저장하고 원인 안내

실제 Windows PC 와 실제 프린터에서는 아직 확인하지 못했다(10.2 절).

> ***AI 활용*** *— 테스트 코드와 GUI 자동 조작 스크립트는 개발 단계마다 AI 가 작성해 실행했고, 결과 해석은 직접 확인했다.*

10. 결론

필수 요구사항 25 개와 확장 기능 19 개를 모두 구현해 개발 목표를 달성했으며, 특히 문제에 없던 **시재 관리**까지 더해 식당의 하루 영업을 시재 개시부터 일일 마감까지 한 프로그램에서 처리할 수 있게 했다.

10.1 성과

- **문제 분석:** 환불 반영 시점, 분할 결제와 할인, 재고 구조처럼 문제에 정해지지 않은 부분을 찾아 대안을 비교한 뒤 결정했다.
- **정확성:** 금액은 정수로 계산하고 끝전은 마지막 결제에서 정리해, 분할 결제·부분 환불·시재에서도 합계가 1 원도 어긋나지 않는다.
- **안정성:** 결제·환불·테이블 분할은 트랜잭션으로 묶었고, 잘못된 입력 21 가지 유형에서 프로그램이 멈추지 않았다.
- **검증:** 업무 규칙을 59 개 자동 테스트로 지켜, 기능을 추가할 때마다 기존 기능이 깨지지 않았는지 바로 확인했다.
- **현장성:** 옵션·요청사항, 주방 주문서, 부분 환불, 시재 정산, exe 배포까지 실제 매장에서 필요한 흐름을 갖췄다.

10.2 한계

- 실제 Windows PC 와 실제 프린터에서는 확인하지 못했다(Wine 환경과 인쇄 실패 경로만 검증).

- 카드 결제는 실제 승인 없이 결제수단만 기록한다(VAN 연동은 인증 필요). 계좌이체도 입금 확인은 직원이 한다.
- PC 한 대에서만 동작한다. 주방·카운터 등 여러 기기가 데이터를 공유하지 못한다.
- 주방에 이미 보낸 메뉴를 "항목 삭제"로 지우면 다음 주문서에 취소가 표시되지 않는다(수량 감소는 표시됨).
- 예약 주문에는 메뉴 옵션을 붙일 수 없다.

10.3 향후 발전 방향

- 같은 공유기 안에서 주방 화면·주문 화면을 다른 PC 로 분리한다(LAN 공유 DB).
- 삭제한 항목도 주방 주문서에 취소로 표시하고, 예약 주문에 옵션을 추가한다.
- 최근 판매량을 바탕으로 재료 발주량을 추천한다(데이터가 충분히 쌓인 뒤 검증).
- 테이블 오더(손님 주문) 화면을 추가한다.

> ***AI 활용*** *— 결론은 프롬프트 "...지금 어떤게 되어 있는지랑 필수 과제 + 확장 과제 어떻게 되어 있는지 정리해주고 보고서 양식은 올린 파일과 동일하게 진행"에 대해 AI 가 개발 과정과 실험 결과를 바탕으로 초안을 작성했다.*

11. 느낀 점

"돈이 맞는가"가 가장 어려운 문제였다. 처음에는 주문과 결제 화면을 만드는 것이 핵심이라고 생각했지만, 실제로는 할인을 나눠 내거나 일부만 환불할 때 1 원도 틀리지 않게 하는 규칙을 정하는 데 가장 많은 시간이 들었다. 시재 점검을 추가하면서 POS 는 결국 장부라는 것을 알게 되었다.

범위를 정하는 것도 설계다. 편의점 모드나 발주 추천처럼 해 보고 싶은 기능이 많았지만, 과제 요구사항과 맞는지, 결과가 믿을 만한지를 기준으로 하지 않을 기능을 고른 것이 오히려 완성도를 높였다.

AI 활용에 대한 회고. AI 와 함께 진행해 짧은 시간에 큰 프로그램을 만들 수 있었지만, 어떤 기능을 넣고 뺄지, 어떤 규칙이 현장에 맞는지는 내가 판단해야 했다. 결과가 이상해 보일 때마다 질문하면서 원리를 확인했고, 그 과정에서 가장 많이 배웠다.

개선하고 싶은 부분. 다음에는 업무 규칙을 먼저 표로 정리하고 테스트를 직접 작성한 뒤 구현을 맡기는 순서로 진행해, 스스로 설계한 부분의 비중을 늘리고 싶다.

> ***AI 활용*** *— 프롬프트 "...보고서 양식은 올린 파일과 동일하게 진행"로 AI 가 대화 기록을 바탕으로 초안을 작성했다.*

12. 기타

12.1 AI 활용 내역

개발 전 과정에서 AI(Anthropic Claude)를 사용했으며, 코드 대부분은 AI 가 작성하고 학습자가 단계마다 결과를 확인하고 질문하며 방향을 정했다. 아래는 실제로 입력한 프롬프트를 진행 순서대로 정리한 것이다.

번호	단계	프롬프트 (원문)	AI 산출 결과
1	문제 분석·구현	이번에 학교 과제로 pos 기기 프로그램 만들껀데 14 페이지 부터 15 페이지의 내용으로 만들꺼야 헤당 이미지의 내용을 따를 필요는 없고 제한 사항이랑 확장 내용은 알아서 추가하는걸로 해보자	Python·Tkinter·SQLite 구조, 요구사항 25 개 구현, 제한사항·확장 기능 정의, 테스트 35 개
2	배포	이거 exe 프로그램으로 진행	PyInstaller exe, exe 용 경로·오류 로그, Wine 에서 빌드·실행 검증
3	점검	현재 제한기능과 확장기능애 대해서 어떻개 되었는지 정리해서 알려줘	제한사항·확장 기능 현황 정리
4	용도 검토	지금 pos 프로그램은 어디에서 가장 유의미 하게 쓸수 았을까	활용처 분석(축제 부스·소규모 매장), 부족한 기능 후보
5	용도 검토	편의점에서 사용할려면 어떤 기능들을 추가해야할까	바코드·유통기한·행사 등 편의점 기능 목록
6	용도 검토	음식점 용도와 편의점 모드를 같이 가져갈때 이점과 편의점 모드만 했을때의 이점	두 방향 비교, 과제 요구사항 기준 권고
7	범위 확정	어쩔수 없다 편의점 모드는 하지 말고 식당 pos 프로그램으로 만 하자	식당 전용 확정, 보강 후보 8 개 제안
8	기능 보강	일단 1~4 까지 하는거랑 6~8 까지 진행 하면 될거 같은데 5 는 너무 확률로 인해서 불안정할거 같아	계좌이체, 옵션·요청사항, 인쇄, 부분 환불, 기간 비교, 복원, 문서 정리
9	점검	지금 제한사항이랑 확장은 다 완성 된건가?	완성도와 남은 빈틈 정리
10	기능 보강	시재점검을 위한 비즈니스 로직 추가해야 할것 같음	시재 개시·입출금·점검·정산, 일일 마감 연동, 테스트 9 개
11	보고서	…지금 어떤게 되어 있는지랑 필수 과제 + 확장 과제 어떻게 되어 있는지 정리해주고 보고서 양식은 올린 파일과 동일하게 진행	이 보고서 초안, 흐름도·구성도

12.2 빌드 및 실행 방법

exe 실행(권장): `KumohPOS.exe` 를 원하는 폴더에 두고 더블클릭한다. 데이터는 exe 와 같은 폴더의 `data\pos.db` 에 저장된다. 관리자 비밀번호 초기값은 `0000` 이다.

```
# Python 으로 실행 (Windows PowerShell)
python main.py
python main.py --demo 60      # 최근 60 일 샘플 데이터 생성 후 실행

# 테스트
python -m unittest discover -s tests -v
```

```
# exe 빌드 (또는 build_exe.bat 더블클릭)
python -m pip install pyinstaller
python -m PyInstaller --noconfirm --clean kumoh_pos.spec
.\dist\KumohPOS.exe
```

시연은 ① `시재` 탭에서 시재 개시 → ② 테이블 선택 후 메뉴·옵션 주문 → ③ 주방 주문서 출력 → ④ 결제(분할·할인) → ⑤ F12 관리자 모드에서 부분 환불·통계 확인 → ⑥ 시재 정산 → ⑦ 일일 마감 순서로 진행한다.